

ARYAN SCHOOL OF ENGINEERING AND MANAGEMENT

Devkota Sadak, Mid Baneshwor, Kathmandu

(Affiliated to Purbanchal University)

A Final Year Project Report on

"HEART FAILURE PREDICTION"

(Project Code: BIT453CO)

Submitted in partial fulfillment of the requirements for the degree of

Bachelor in Information Technology (BIT)

Submitted by:

Anup Karki (380181)

Dipesh Kumar Sharma (380184)

Puskar Sarki (380191)

Submitted to:

Department of Science and Technology

Aryan School of Engineering and Management

June, 2026



ARYAN SCHOOL OF ENGINEERING AND MANAGEMENT

Devkota Sadak, Mid Baneshwor, Kathmandu

(Affiliated to Purbanchal University)

A Final Year Project Report on

"HEART FAILURE PREDICTION"

(Project Code: BIT453CO)

Submitted in partial fulfillment of the requirements for the degree of

Bachelor in Information Technology (BIT)

Submitted by:

Anup Karki (380181)

Dipesh Kumar Sharma (380184)

Puskar Sarki (380191)

Under the Supervision of:

Mr. Sunny Sharma

Submitted to:

Department of Science and Technology

Aryan School of Engineering and Management

June, 2026

SUPERVISOR RECOMMENDATION

I hereby recommend that this project report prepared under my supervision by Anup Karki (380181), Dipesh Kumar Sharma (380184) and Puskar Sarki (380191), entitled **"Heart Failure Prediction"**, in partial fulfillment of the requirements for the degree of Bachelor in Information Technology (BIT) of Purbanchal University, be processed for evaluation.

To the best of my knowledge, the work presented in this report is the original work of the students, and the system described has been designed, implemented and tested by them.

……

Mr. Sunny Sharma
Project Supervisor
Department of Science and Technology
Aryan School of Engineering and Management
Date: ……

CERTIFICATE OF APPROVAL

This is to certify that this project report prepared by Anup Karki (380181), Dipesh Kumar Sharma (380184) and Puskar Sarki (380191), entitled **"Heart Failure Prediction"**, in partial fulfillment of the requirements for the degree of Bachelor in Information Technology (BIT), has been evaluated. In our opinion, it is satisfactory in scope and quality as a project for the required degree.

Evaluation Committee

……	……
Mr. Sunny Sharma	**……**
Supervisor	Head of Department
……	……
……	**……**
Internal Examiner	External Examiner

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our supervisor, **Mr. Sunny Sharma**, for the valuable guidance, constructive feedback and continuous support throughout this project. The suggestions received during the proposal, mid-term and final phases helped us shape both the machine learning model and the web application.

We are thankful to the Department of Science and Technology, Aryan School of Engineering and Management, for providing the opportunity, resources and environment needed to carry out this project. We also thank the faculty members who reviewed our work and offered suggestions during the presentations.

We acknowledge the creators of the Heart Failure Clinical Records dataset and the maintainers of the UCI Machine Learning Repository, whose openly available data made this work possible.

Lastly, we would like to thank our friends and family for their continuous motivation and support during the project journey.

Anup Karki (380181)

Dipesh Kumar Sharma (380184)

Puskar Sarki (380191)

ABSTRACT

Heart failure is a leading cause of hospitalization and death worldwide, and in low-resource settings such as Nepal, early identification of high-risk patients is limited by manual, experience-driven assessment. This project presents the Heart Failure Prediction (HFP) system, a web application that estimates a patient's risk of a heart-failure death event from twelve routinely collected clinical attributes, including age, ejection fraction, serum creatinine, serum sodium and comorbidities.

The model was developed on the UCI Heart Failure Clinical Records dataset, expanded from 299 real patients to 2000 records by controlled jittering, with a patient-group-aware train/test split to prevent information leakage. Six classifiers were compared: Logistic Regression, Support Vector Machine, Decision Tree, Random Forest, XGBoost and Gradient Boosting. On 589 held-out records, the deployed XGBoost model achieved an accuracy of 75.0%, precision of 69.9%, recall of 64.8%, F1-score of 0.673 and the highest ROC-AUC of 0.848. Random Forest obtained a slightly higher F1-score (0.698); XGBoost was retained for its equal discrimination ability and its native, exact SHAP explanations.

The system is implemented with a FastAPI backend, a React frontend and an SQLite database. Users can register, submit clinical data, and receive a risk status, probability, risk level, the five factors that most influenced their prediction and rule-based health suggestions, together with a downloadable PDF report and a history of previous predictions. The system was verified with 19 automated unit and integration tests and 14 browser-based system test cases.

Keywords: heart failure, machine learning, XGBoost, explainable AI, SHAP, clinical decision support

TABLE OF CONTENTS

SUPERVISOR RECOMMENDATION ..... **i**

CERTIFICATE OF APPROVAL ..... **ii**

ACKNOWLEDGEMENT ..... **iii**

ABSTRACT ..... **iv**

LIST OF ABBREVIATIONS ..... **viii**

LIST OF FIGURES ..... **ix**

LIST OF TABLES ..... **xi**

CHAPTER 1: INTRODUCTION **1**

1.1 Introduction 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Scope and Limitation 2

1.4.1 Scope 2

1.4.2 Limitation 2

1.5 Development Methodology 3

1.6 Report Organization 4

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW **5**

2.1 Background Study 5

2.1.1 Clinical Attributes Used for Prediction 5

2.1.2 Heart Failure Prediction as a Classification Problem 7

2.1.3 Ensemble Learning and Gradient Boosting 7

2.1.4 Explainable Machine Learning 7

2.1.5 Web Application Architecture 8

2.2 Literature Review 8

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN **10**

3.1 System Analysis 10

3.1.1 Requirement Analysis 10

3.1.2 Feasibility Analysis 14

3.1.3 Object Modelling using Class and Object Diagrams 15

3.1.4 Dynamic Modelling using State and Sequence Diagrams.....17
3.1.5 Process Modelling using Activity Diagrams.....19
3.2 System Design.....20
3.2.1 Refinement of Class, Object, State, Sequence and Activity Diagrams.....21
3.2.2 Component Diagram.....26
3.2.3 Deployment Diagram.....27
3.3 Algorithm Details.....28
3.3.1 Data Preparation and Preprocessing.....28
3.3.2 Logistic Regression and Support Vector Machine.....29
3.3.3 Decision Tree Classifier.....29
3.3.4 Random Forest Classifier.....30
3.3.5 XGBoost Classifier (Deployed Model).....31
3.3.6 Explaining Predictions with TreeSHAP.....32
3.3.7 Risk Levels and Suggestion Rules.....33
3.3.8 Evaluation Metrics.....33
CHAPTER 4: IMPLEMENTATION AND TESTING.....34
4.1 Implementation.....34
4.1.1 Tools Used.....34
4.1.2 Implementation Details of Modules.....34
4.2 Testing.....37
4.2.1 Test Cases for Unit Testing.....38
4.2.2 Test Cases for System Testing.....38
4.3 Result Analysis.....39
4.3.1 Model Comparison.....39
4.3.2 Feature Importance.....41
4.3.3 System Output and Performance.....42
CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS.....44
5.1 Conclusion.....44
5.2 Future Recommendations.....44
REFERENCES.....46
BIBLIOGRAPHY.....48

APPENDICES **49**

Appendix A: Screenshots 49

Appendix B: Source Code 56

B.1 Prediction Module (backend/app/predictor.py) 56

B.2 Prediction Endpoint (backend/app/main.py) 57

B.3 Suggestion Module (backend/app/suggestions.py) 57

B.4 Model Definitions (ml/train.py) 58

B.5 Prediction Form Submission (frontend/src/pages/PredictionForm.jsx) 59

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
AUC	Area Under the Curve
BIT	Bachelor in Information Technology
CPK	Creatinine Phosphokinase
CSV	Comma-Separated Values
CT	Computed Tomography
CV	Cross-Validation
DL	Deep Learning
EF	Ejection Fraction
EHR	Electronic Health Record
FN	False Negative
FP	False Positive
HF	Heart Failure
HFP	Heart Failure Prediction
HTTP	Hypertext Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
JWT	JSON Web Token
LIME	Local Interpretable Model-agnostic Explanations
ML	Machine Learning
PBKDF2	Password-Based Key Derivation Function 2
PDF	Portable Document Format
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SPA	Single Page Application
SQL	Structured Query Language
SVM	Support Vector Machine
TN	True Negative
TP	True Positive
UCI	University of California, Irvine
UI	User Interface
UML	Unified Modeling Language
XGBoost	Extreme Gradient Boosting

LIST OF FIGURES

Figure 1: Agile Methodology..... 3
Figure 2: Class Distribution of the Original and Expanded Datasets.....6
Figure 3: Correlation Matrix of the Clinical Attributes.....6
Figure 4: Use Case Diagram of the HFP System..... 11
Figure 5: Gantt Chart of the Project Schedule.....15
Figure 6: Class Diagram of the HFP System..... 16
Figure 7: Object Diagram of the HFP System.....17
Figure 8: State Diagram of the HFP System..... 18
Figure 9: Sequence Diagram of the HFP System..... 19
Figure 10: Activity Diagram of the Prediction Process.....20
Figure 11: System Architecture (Block Diagram) of the HFP System.....21
Figure 12: Refined Class Diagram of the HFP System.....22
Figure 13: Refined Object Diagram of the Backend at Run Time.....23
Figure 14: Refined State Diagram of a Prediction Request.....24
Figure 15: Refined Sequence Diagram of the Prediction Flow.....25
Figure 16: Refined Activity Diagram of the Model Training Pipeline.....26
Figure 17: Component Diagram of the HFP System.....27
Figure 18: Deployment Diagram of the HFP System.....28
Figure 19: Decision Tree Classifier.....30
Figure 20: Random Forest Classifier.....31
Figure 21: XGBoost Classifier..... 31
Figure 22: Test-set Performance of the Six Classifiers.....40
Figure 23: Confusion Matrices of the Six Models on the Test Set.....41
Figure 24: ROC Curves of the Six Models.....41
Figure 25: Feature Importance of the XGBoost Model.....42
Figure 26: Login Page..... 49
Figure 27: Sign Up Page..... 50
Figure 28: Reset Password Page.....50
Figure 29: Heart Failure Risk Prediction Form with Field Description.....51

Figure 30: Validation of Invalid Input.....51
Figure 31: Prediction Result Page (Healthy Patient).....52
Figure 32: Prediction Result Page (At-Risk Patient) with Contributing Factors and Suggestions.....53
Figure 33: Prediction History Page.....53
Figure 34: Downloaded PDF Report of a Patient.....54
Figure 35: Interactive API Documentation of the Backend.....55

LIST OF TABLES

Table 1: Clinical Attributes of the Heart Failure Clinical Records Dataset.....5
Table 2: Summary of Reviewed Literature.....9
Table 3: Use Case Description – UC-01: Register Account.....11
Table 4: Use Case Description – UC-02: Login.....12
Table 5: Use Case Description – UC-03: Reset Password.....12
Table 6: Use Case Description – UC-04: Submit Health Data for Prediction.....12
Table 7: Use Case Description – UC-05: View Prediction Result.....12
Table 8: Use Case Description – UC-06: Download PDF Report.....13
Table 9: Use Case Description – UC-07: View Prediction History.....13
Table 10: Use Case Description – UC-08: Logout.....13
Table 11: XGBoost Hyperparameter Grid and Selected Values.....32
Table 12: Risk Level Thresholds.....33
Table 13: Rules of the Suggestion Engine.....33
Table 14: Tools and Technologies Used.....34
Table 15: Database Schema.....36
Table 16: REST API Endpoints.....37
Table 17: Unit and Integration Test Cases.....38
Table 18: System Test Cases.....38
Table 19: Performance Comparison of the Machine Learning Models.....39
Table 20: Sample Predictions of the Deployed System.....42

CHAPTER 1: INTRODUCTION

1.1 Introduction

Heart failure (HF) is a chronic, progressive condition in which the heart cannot pump enough blood to meet the body's needs. It affects tens of millions of people worldwide and remains a major cause of hospitalization, poor quality of life and death [1]. Many deaths could be delayed if high-risk patients were identified early and followed up more closely, but such identification usually depends on a clinician manually interpreting several isolated laboratory values and clinical signs.

Routine clinical records already contain strong signals of risk. Studies on patients admitted with heart failure have shown that age, ejection fraction, serum creatinine, serum sodium and comorbidities such as anaemia and high blood pressure are associated with survival [2], and that machine learning models can predict a patient's outcome from these attributes [3]. The Heart Failure Prediction (HFP) system developed in this project turns this idea into a usable tool: a web application where a user enters twelve clinical values and immediately receives a risk prediction, an explanation of the factors behind it, health suggestions and a printable PDF report.

The prediction engine is an XGBoost classifier selected after comparing six machine learning algorithms. It is served by a FastAPI backend and accessed through a React frontend, with an SQLite database storing user accounts and prediction history.

1.2 Problem Statement

In Nepal, as in many low-resource settings, heart failure care is largely reactive. Patient information is collected on paper or in fragmented hospital systems, and risk assessment depends on the experience of the individual clinician. There is no simple, data-driven tool that combines a patient's routine clinical values into a single, explainable risk estimate. As a result:

- High-risk patients may not be identified until their symptoms worsen.
- Assessments of the same patient can differ between clinicians, because they are subjective.
- Patients have no easy way to understand which of their clinical values contribute most to their risk.

- Clinicians cannot easily keep a structured record of previous risk assessments.

This project addresses these problems by building an automated, explainable heart failure risk prediction system that works from attributes already recorded in routine care.

1.3 Objectives

The main objective of this project is to develop a machine learning-based web application that predicts the risk of heart failure from clinical data to support early detection and decision-making. The specific objectives are:

- To train and compare machine learning models that predict heart-failure risk from twelve clinical attributes, and deploy the most suitable model.
- To explain each prediction by identifying the clinical factors that increased or decreased the risk.
- To provide a secure web application with registration, prediction, history, rule-based suggestions and downloadable PDF reports.

1.4 Scope and Limitation

1.4.1 Scope

- **Decision support:** the system provides a risk estimate that can help clinicians and patients decide when closer follow-up is needed.
- **Data handling:** the system validates clinical input and stores each user's health data and predictions securely for later review.
- **Model development:** the project covers data preparation, training and comparison of six classifiers, evaluation on held-out data and export of the selected model.
- **Explainability:** each prediction lists its five most influential factors using SHAP values, and the system gives rule-based suggestions for abnormal clinical values.
- **Reporting:** users can download a PDF report of the prediction and view their prediction history.

1.4.2 Limitation

- **Small source dataset:** the model is derived from 299 real patients; the 2000-record training set is a synthetic expansion of these patients and cannot add genuinely new clinical information.

- **No external validation:** the data comes from a single hospital in Pakistan, and the model has not been validated on patients from Nepali hospitals.
- **Limited risk factors:** only twelve attributes are used; important factors such as ECG findings, natriuretic peptides and medication history are not considered.
- **Follow-up time dependence:** the follow-up period is the strongest predictor in the dataset, but it is partly a consequence of the outcome rather than a cause, which limits clinical interpretation.
- **Not a diagnostic device:** predictions support but do not replace professional medical judgement, and the system is not integrated with live hospital systems.

1.5 Development Methodology

The project followed the **Agile** development methodology. Machine learning projects suit an iterative approach because the dataset, model performance and user interface requirements change as the work progresses. Development was divided into short iterations (sprints). Each sprint was planned, designed, developed, tested and reviewed with the supervisor before the next one started, as shown in Figure 1.

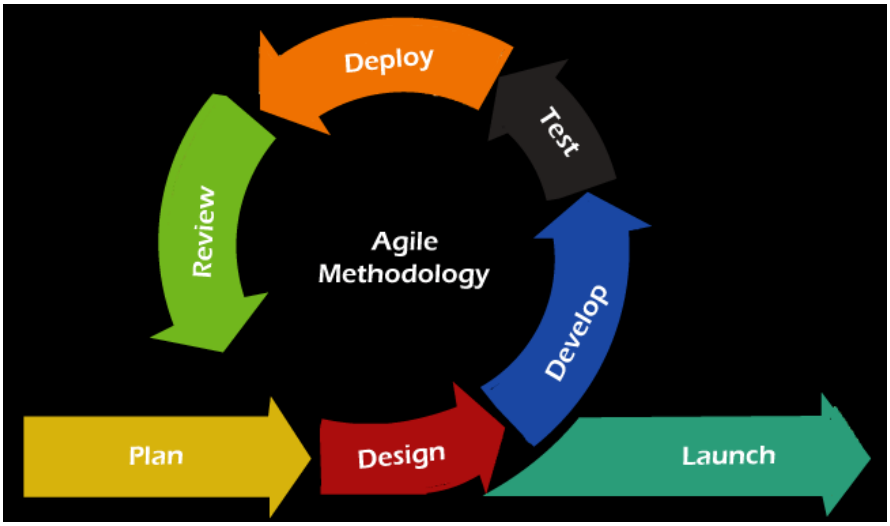


Figure 1: Agile Methodology

The phases of each iteration were applied to the project as follows:

- **Plan:** define the goals of the sprint, such as preparing the dataset, adding the prediction API or building the result page, and prioritize the related tasks.
- **Design:** prepare UML diagrams, the database schema, the API contract and the user interface layout for the features of the sprint.

- **Develop:** implement the features, for example the training script, the FastAPI endpoints or the React pages.
- **Test:** run automated unit and integration tests and manual checks of the user interface, and fix the defects found.
- **Deploy:** run the updated backend and frontend locally so the supervisor and team members can try the new features.
- **Review:** collect feedback, compare the model metrics with the previous iteration and decide on improvements.
- **Launch:** deliver the final, fully tested system for the final defense and demonstration.

The main iterations were: (1) requirement study and dataset analysis, (2) data preparation and model comparison, (3) backend API with authentication and prediction, (4) frontend pages and PDF report, and (5) explainability, suggestions, testing and documentation.

1.6 Report Organization

This report is organized into five chapters:

- **Chapter 1: Introduction** presents the problem, objectives, scope, limitations and the development methodology of the project.
- **Chapter 2: Background Study and Literature Review** explains the clinical and machine learning concepts used in the project and reviews related research.
- **Chapter 3: System Analysis and Design** describes the requirements, feasibility analysis, object, dynamic and process models, the refined design, component and deployment diagrams, and the algorithms used.
- **Chapter 4: Implementation and Testing** describes the tools and the implementation of each module, the unit and system test cases, and the analysis of the results.
- **Chapter 5: Conclusion and Future Recommendations** summarizes the work and suggests future improvements.

The report ends with the references, bibliography and appendices containing screenshots of the system and selected source code.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

2.1.1 Clinical Attributes Used for Prediction

The system uses the twelve attributes of the Heart Failure Clinical Records dataset [4], collected by Ahmad et al. from 299 heart failure patients at the Faisalabad Institute of Cardiology and the Allied Hospital in Faisalabad, Pakistan, between April and December 2015 [2]. All patients had left ventricular systolic dysfunction and were aged 40 to 95 years; 194 were men and 105 were women. The target attribute, DEATH_EVENT, records whether the patient died during the follow-up period (96 patients, 32.1%). Table 1 describes the attributes as they are used in the HFP input form.

Table 1: Clinical Attributes of the Heart Failure Clinical Records Dataset

Attribute	Meaning in the System	Unit / Values	Range (mean)
age	Age of the patient	years	40–95 (60.8)
sex	Biological sex	1 = male, 0 = female	65% male
anaemia	Decrease of red blood cells or haemoglobin	0 / 1	43% yes
creatinine_phosphokinase	Level of the CPK enzyme in the blood	mcg/L	23–7861 (582)
diabetes	Patient has diabetes	0 / 1	42% yes
ejection_fraction	Percentage of blood leaving the heart at each contraction	%	14–80 (38.1)
high_blood_pressure	Patient has hypertension	0 / 1	35% yes
platelets	Platelets in the blood	kiloplatelets/ mL	25,100–850,000 (263,358)
serum_creatinine	Level of serum creatinine (kidney function)	mg/dL	0.5–9.4 (1.39)
serum_sodium	Level of serum sodium	mEq/L	113–148 (136.6)
smoking	Patient smokes	0 / 1	32% yes
time	Follow-up period	days	4–285 (130.3)
DEATH_EVENT	Patient died during follow-up (target)	0 / 1	32.1% yes

Several attributes have direct clinical meaning for heart failure. A low **ejection fraction** (below 40%) indicates reduced pumping function of the left ventricle. An elevated **serum creatinine** reflects impaired kidney function, which commonly accompanies and worsens heart failure. Low **serum sodium** (hyponatraemia) is associated with fluid overload and poor outcomes. These clinical reference ranges are also used by the rule-based suggestion module described in Section 3.3.7.

Figure 2 shows the class distribution of the original and the expanded datasets. Both are imbalanced, with about two survivors for every death event, so the models must be evaluated with precision, recall and F1-score rather than accuracy alone. Figure 3 shows the correlation between the attributes: the death event is most strongly (negatively) correlated with follow-up time, and positively correlated with serum creatinine and age, while ejection fraction and serum sodium are negatively correlated with it.

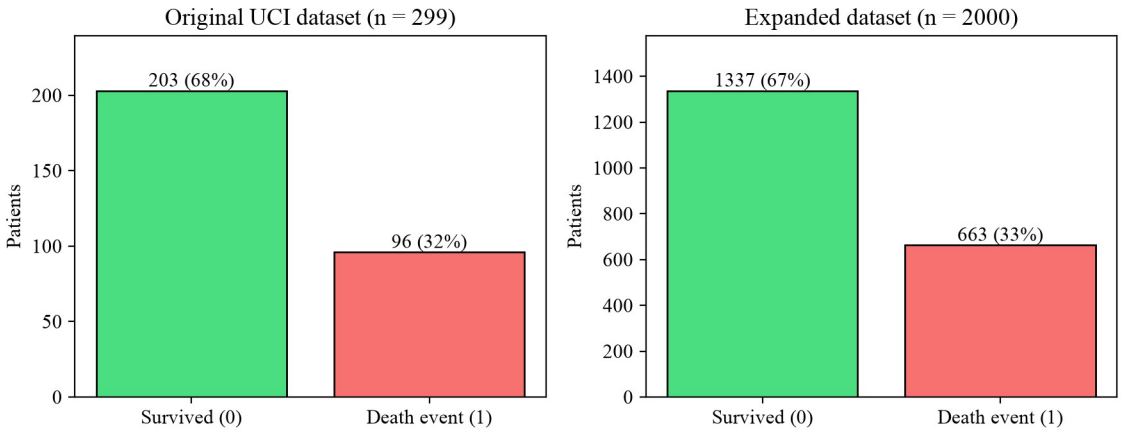


Figure 2: Class Distribution of the Original and Expanded Datasets

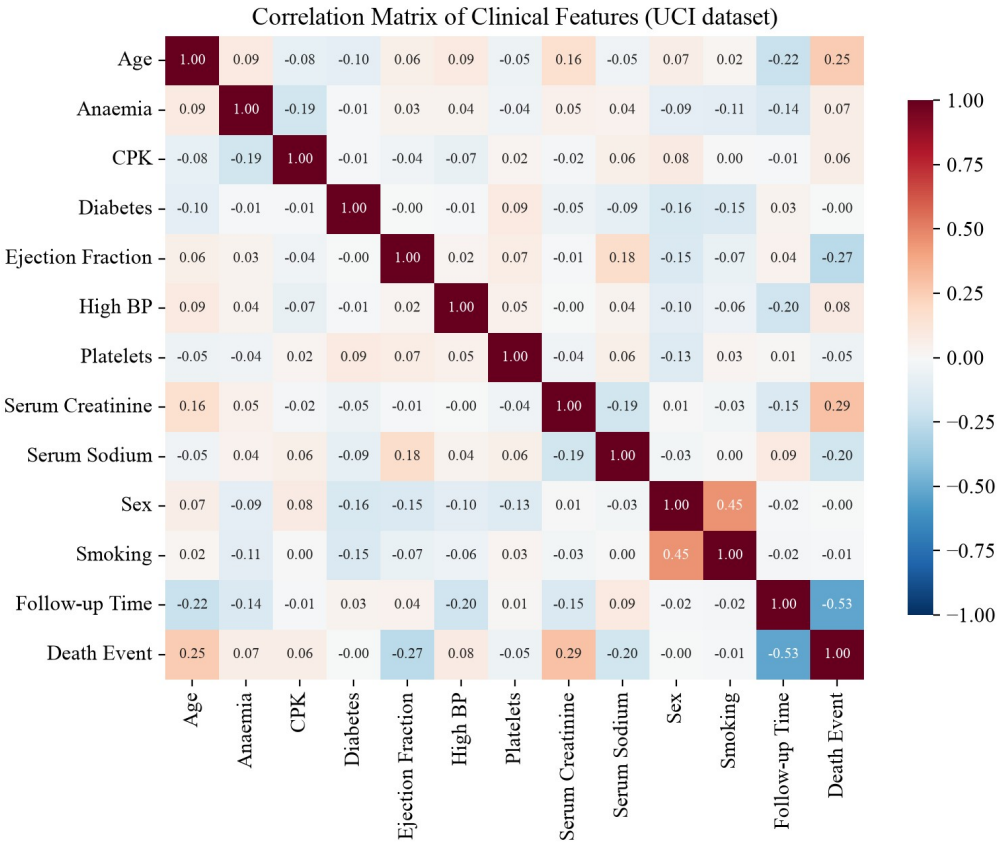


Figure 3: Correlation Matrix of the Clinical Attributes

2.1.2 Heart Failure Prediction as a Classification Problem

Predicting whether a patient will experience a death event is a **supervised binary classification** problem: each patient is described by a feature vector of twelve attributes and labelled 1 (death event) or 0 (survived). A classifier learns a function that maps the feature vector to the probability of class 1. In the HFP system, this probability is shown to the user as the *risk probability*, and a threshold of 0.5 converts it into the status *At Risk* or *Healthy*.

Because the source dataset is small, two practical issues shape the design. First, **class imbalance** biases models towards the majority class. The project handles this by weighting the positive class during training and by selecting models on F1-score. Second, **evaluation leakage** can make results look better than they are. The expanded dataset contains several synthetic copies of each real patient, so a random split would place copies of the same patient in both the training and test sets. The project prevents this by splitting the data by patient group (Section 3.3.1).

2.1.3 Ensemble Learning and Gradient Boosting

Tree-based ensemble methods combine many decision trees into a stronger model. **Bagging** methods such as Random Forest train trees independently on bootstrap samples and average their predictions, which reduces variance [5]. **Boosting** methods build trees sequentially, each one correcting the errors of the previous ones. Gradient boosting formalizes this as gradient descent on a loss function [6], and XGBoost adds regularization, efficient split finding and parallel computation to this framework [7]. These properties make boosted trees accurate on small, tabular clinical datasets such as the one used in this project.

2.1.4 Explainable Machine Learning

A risk score is more useful in a clinical setting when it comes with an explanation. **SHAP** (SHapley Additive exPlanations) attributes a model's output to its input features using Shapley values from cooperative game theory, so that the contributions of all features add up to the prediction [8]. For tree ensembles, TreeSHAP computes these values exactly and efficiently [9]. The HFP system uses XGBoost's built-in TreeSHAP implementation to show each patient the five factors that most increased or decreased their predicted risk.

2.1.5 Web Application Architecture

The HFP system follows a client-server architecture. The React frontend is a single page application (SPA) that runs in the browser and communicates with the backend through a REST API that exchanges JSON. The FastAPI backend validates every request against typed schemas, authenticates users with signed JSON Web Tokens (JWT) and runs the trained model. Passwords are stored only as salted PBKDF2 hashes. This separation allows the model, the API and the user interface to be developed and tested independently.

2.2 Literature Review

Early computational approaches to cardiac diagnosis were based on statistical models. Detrano et al. developed a probability algorithm for diagnosing coronary artery disease and validated it on patient groups from Cleveland, Hungary, Switzerland and Long Beach [10]. The study showed that statistical models built from routine clinical attributes could estimate disease probability across different populations. Its data later became one of the most widely used benchmark datasets for heart disease prediction research.

With the growth of machine learning, researchers began to apply non-linear classifiers to cardiac data. Gudadhe et al. proposed a decision support system for heart disease based on Support Vector Machines and Artificial Neural Networks, showing that these models can capture complex patterns in patient attributes [11]. Jabbar et al. combined associative classification with a genetic algorithm to discover compact, accurate rules for heart disease prediction [12]. Mohan et al. proposed a hybrid random forest with a linear model (HRFLM) and showed that combining different techniques improved prediction accuracy over individual classifiers [13]. Together, these studies suggest that ensemble and hybrid tree-based models are well suited to tabular cardiac data, which motivated the comparison of tree ensembles in this project.

Closest to this project, Ahmad et al. collected the heart failure dataset used here and applied survival analysis to it. They found that age, renal dysfunction (serum creatinine), blood pressure, ejection fraction and anaemia were significant risk factors for mortality [2]. Chicco and Jurman then applied several machine learning classifiers to the same dataset and found that Random Forest performed best. They also showed that a model using only serum creatinine and ejection fraction could predict survival nearly as well as one using all features [3]. The feature importance obtained in this project (Section 4.3)

agrees with this finding: after follow-up time, serum creatinine and ejection fraction are the most important features of the XGBoost model.

Deep learning has extended disease prediction to other data types. Lakshmanaprabu et al. developed an optimal deep learning model for classifying lung cancer from CT images, showing the strength of deep networks on medical images [14]. Alaa and van der Schaar proposed an interpretable deep learning model that forecasts individual disease trajectories over time and explains its predictions to clinicians [15]. Their emphasis on interpretability is shared by the SHAP-based explanations used in this project [8], [9]. However, deep learning offers little advantage on a small tabular dataset such as the one used here, where tree ensembles remain the stronger choice.

Table 2: Summary of Reviewed Literature

Ref.	Approach	Data	Relevance to HFP
[10]	Statistical probability algorithm	Coronary artery disease patients, four centres	Clinical attributes can estimate cardiac risk
[11]	SVM and ANN decision support	Heart disease records	Non-linear classifiers for cardiac data
[12]	Associative classification + genetic algorithm	Heart disease records	Rule-based, interpretable prediction
[13]	Hybrid random forest with linear model	Heart disease records	Hybrid and ensemble models improve accuracy
[2]	Survival analysis	299 heart failure patients	Source of the dataset; key risk factors
[3]	Machine learning classifiers, feature ranking	Same 299 patients	Serum creatinine and EF are most predictive
[14]	Optimal deep learning	Lung CT images	Deep learning for medical imaging
[15]	Interpretable deep learning	Longitudinal patient data	Interpretable, individualized prediction

Review of the existing system. In current practice, heart failure risk is assessed manually. Clinicians interpret isolated test results using general clinical guidelines, and patient records are often incomplete or spread across paper files and local hospital systems. Assessments are subjective, usually happen after symptoms worsen, and do not give patients an explanation of their risk. The HFP system addresses these gaps. It standardizes data entry, produces a consistent, data-driven risk estimate, explains the main contributing factors, and keeps a history of previous assessments.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

System analysis identifies what the HFP system must do and models its structure, behaviour and processes before implementation. The analysis is based on the problems identified in the existing manual process (Section 2.2) and on the needs of the intended users: patients, clinicians, data analysts and researchers.

3.1.1 Requirement Analysis

i. Functional Requirements

The functional requirements of the system are captured in the use case diagram in Figure 4. The primary actor is the user (a patient or clinician). The secondary actor is the prediction engine, the trained XGBoost model that the system calls to score health data. Submitting health data always includes validation and displaying the result, while viewing the contributing factors and suggestions extends the result view.

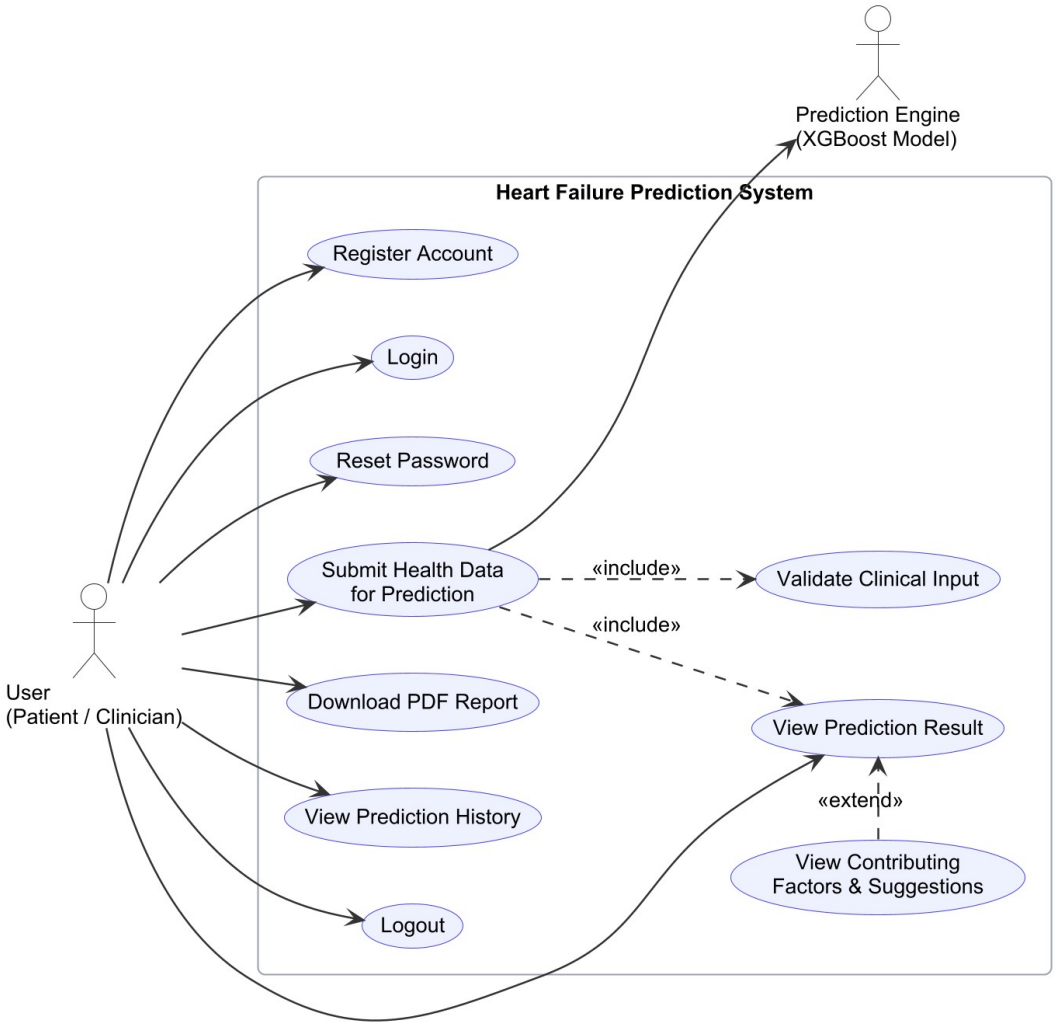


Figure 4: Use Case Diagram of the HFP System

The use cases are described in Tables 3 to 10.

Table 3: Use Case Description – UC-01: Register Account

Use Case ID	UC-01
Use Case Name	Register Account
Actor	User
Description	A new user creates an account to access the prediction features.
Pre-condition	The user is not logged in.
Main Flow	1. User opens the Sign Up page. 2. User enters name, email, phone number, address and password. 3. System validates the fields (valid email, 7–15 digit phone, password of at least 8 characters). 4. System stores the user with a salted PBKDF2 password hash. 5. System issues an access token and opens the prediction form.
Alternate Flow	3a. Invalid field: the system shows a message next to the field. 4a. Email already registered: the system shows “An account with this email already exists”.
Post-condition	A new account exists and the user is logged in.

Table 4: Use Case Description – UC-02: Login

Use Case ID	UC-02
Use Case Name	Login
Actor	User
Description	A registered user signs in with email and password.
Pre-condition	The user has an account.
Main Flow	1. User enters email and password. 2. System finds the account and verifies the password hash. 3. System returns a signed JWT valid for 12 hours, stored in the browser. 4. System opens the prediction form.
Alternate Flow	2a. Wrong email or password: the system shows “Invalid email or password”.
Post-condition	The user is authenticated.

Table 5: Use Case Description – UC-03: Reset Password

Use Case ID	UC-03
Use Case Name	Reset Password
Actor	User
Description	A user who forgot the password sets a new one.
Pre-condition	The user has an account and is not logged in.
Main Flow	1. User selects “Forgot password?” on the login page. 2. User enters the registered email, the registered phone number and a new password twice. 3. System verifies that the email and phone number match the same account. 4. System stores the new password hash and returns to the login page with a success message.
Alternate Flow	2a. Passwords do not match: an error is shown. 3a. Email and phone do not match: an error is shown.
Post-condition	The user can log in with the new password.

Table 6: Use Case Description – UC-04: Submit Health Data for Prediction

Use Case ID	UC-04
Use Case Name	Submit Health Data for Prediction
Actors	User (primary), Prediction Engine (secondary)
Description	The user enters twelve clinical values and requests a heart failure risk prediction.
Pre-condition	The user is logged in.
Main Flow	1. User fills the General, Medical and Habit sections of the form (help text is shown for the focused field). 2. User clicks “Predict Risk”. 3. System validates every field on the client (includes UC: Validate Clinical Input). 4. Backend authenticates the request and validates the values again. 5. Prediction Engine returns the risk probability and SHAP contributions. 6. System generates suggestions, stores the health data and prediction, and shows the result (includes UC-05).
Alternate Flow	3a. Missing or out-of-range value: the field is highlighted with a message and nothing is sent. 4a. Expired token: the user is returned to the login page.
Post-condition	A health data record and its prediction are stored for the user.

Table 7: Use Case Description – UC-05: View Prediction Result

Use Case ID	UC-05
Use Case Name	View Prediction Result
Actor	User

Description	The user views the status, probabilities and risk level of a prediction, with the contributing factors and suggestions (extension).
Pre-condition	A prediction has been made or selected from history.
Main Flow	1. System shows patient details, the status (Healthy / At Risk), the risk level and the healthy and risk probabilities. 2. System lists the five most influential factors, each marked as increasing or decreasing the risk. 3. System lists rule-based suggestions for the patient's abnormal values.
Post-condition	The user understands the predicted risk and its main causes.

Table 8: Use Case Description – UC-06: Download PDF Report

Use Case ID	UC-06
Use Case Name	Download PDF Report
Actor	User
Description	The user saves a PDF report of a prediction.
Pre-condition	A prediction result is displayed.
Main Flow	1. User clicks “Download PDF Report”. 2. System generates a report containing the result, patient details, all clinical inputs, contributing factors and suggestions. 3. Browser downloads the file HFP_Report_<name>_<id>.pdf.
Post-condition	The report is saved on the user's device.

Table 9: Use Case Description – UC-07: View Prediction History

Use Case ID	UC-07
Use Case Name	View Prediction History
Actor	User
Description	The user reviews previous predictions.
Pre-condition	The user is logged in.
Main Flow	1. User opens the History page. 2. System lists the user's predictions, newest first, with date, status, risk level and key values. 3. User selects “View” to reopen a past result (UC-05).
Alternate Flow	2a. No predictions yet: the system shows a link to the prediction form.
Post-condition	Only the user's own records are shown.

Table 10: Use Case Description – UC-08: Logout

Use Case ID	UC-08
Use Case Name	Logout
Actor	User
Description	The user ends the session.
Pre-condition	The user is logged in.
Main Flow	1. User clicks “Logout”. 2. System removes the token from the browser and opens the login page.
Post-condition	Protected pages require logging in again.

ii. Non-Functional Requirements

- **Accuracy:** the model must discriminate well between high- and low-risk patients, with a balance between precision and recall. It is evaluated on patients not seen during training (the deployed model achieves a ROC-AUC of 0.848).

- **Performance:** a prediction must be returned quickly enough for interactive use. The target is under one second; the measured server time is about 2 ms (Section 4.3).
- **Security:** passwords are stored only as salted PBKDF2-SHA256 hashes (310,000 iterations). API access requires a signed, expiring JWT, and each user can access only their own records.
- **Usability:** the interface groups the inputs into General, Medical and Habit sections, shows help text and units for every field, highlights invalid values and works on mobile screens.
- **Interpretability:** every prediction is accompanied by its main contributing factors and plain-language suggestions.
- **Reliability and maintainability:** inputs are validated on both the client and the server. The model, API and interface are separate modules covered by automated tests.
- **Portability:** the system runs in any modern browser (Chrome, Firefox, Edge) and the server runs on Windows, macOS or Linux with Python 3.11 or newer.

3.1.2 Feasibility Analysis

A feasibility study was carried out to confirm that the proposed system was practical to build and use.

i. Technical Feasibility

The system is built entirely with mature, well-documented open-source technologies: Python with scikit-learn and XGBoost for machine learning, FastAPI for the backend, React for the frontend and SQLite for storage. The team had prior experience with Python and JavaScript, and the trained model is small (under 200 KB) and needs no special hardware. A laptop with 8 GB RAM and an Intel Core i5 class processor is enough for both training and serving. The main technical challenge was the small size of the available dataset, which was addressed with careful data expansion and leakage-free evaluation. The project is therefore technically feasible.

ii. Operational Feasibility

The system requires only a web browser. Users enter values they already find in routine clinical reports, and the result appears within a second, with an explanation and a printable report. This fits into a consultation without disrupting existing work. The interface uses familiar web conventions and contextual help, so minimal training is

needed. The system is therefore operationally feasible as a decision support tool, provided that users understand that it does not replace clinical diagnosis.

iii. Economic Feasibility

All software used in development and deployment is free and open source, so there are no licensing costs. Development was done on the team's own computers, and the system can run on a single low-cost server or an existing hospital machine. The main investment is the team's development time. Against this low cost, earlier identification of high-risk patients can reduce emergency admissions and support better use of limited clinical resources. The project is therefore economically feasible.

iv. Schedule Feasibility

The project was planned over six months, from January to June 2025, as shown in the Gantt chart in Figure 5. Overlapping phases and the iterative Agile approach allowed testing to begin while coding continued, and documentation to start before testing finished. All phases were completed within the academic schedule, so the project was feasible in terms of time.

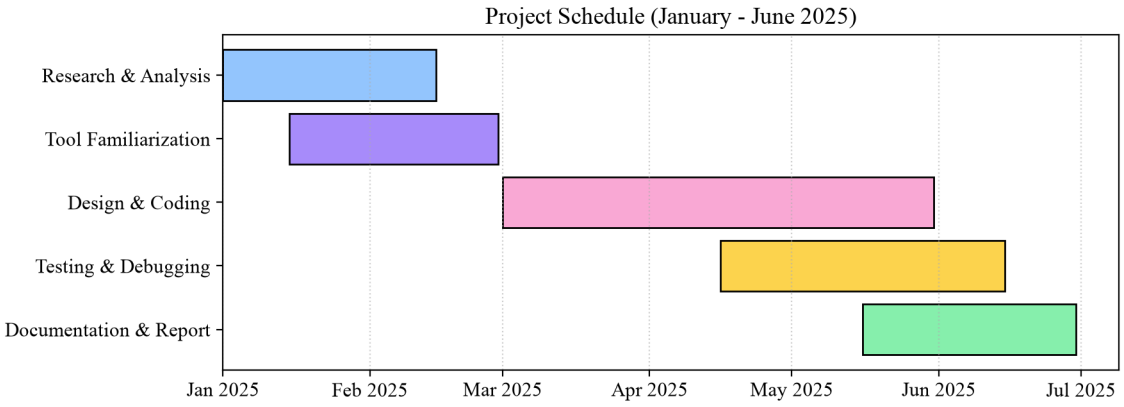


Figure 5: Gantt Chart of the Project Schedule

3.1.3 Object Modelling using Class and Object Diagrams

The analysis class diagram in Figure 6 shows the main domain concepts and their relationships. A **User** submits any number of **HealthData** records. Each HealthData record produces exactly one **Prediction**, which is composed of one to five contributing **Factor** objects and can be rendered as a **Report**. The **PredictionModel** uses HealthData to create Predictions and their explanations.

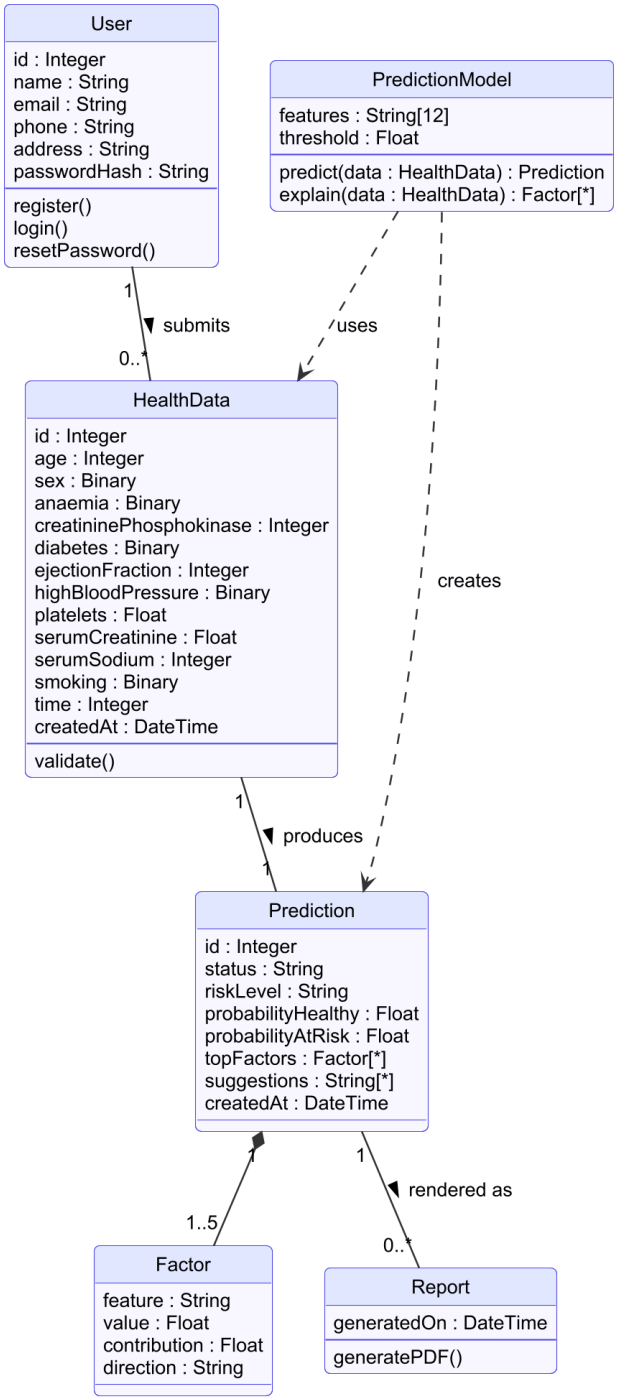


Figure 6: Class Diagram of the HFP System

The object diagram in Figure 7 shows a snapshot of these classes at run time, using the example patient whose prediction is shown in Appendix A: a 60-year-old male patient with an ejection fraction of 38% received a *Healthy* prediction with a risk probability of 13.96%.

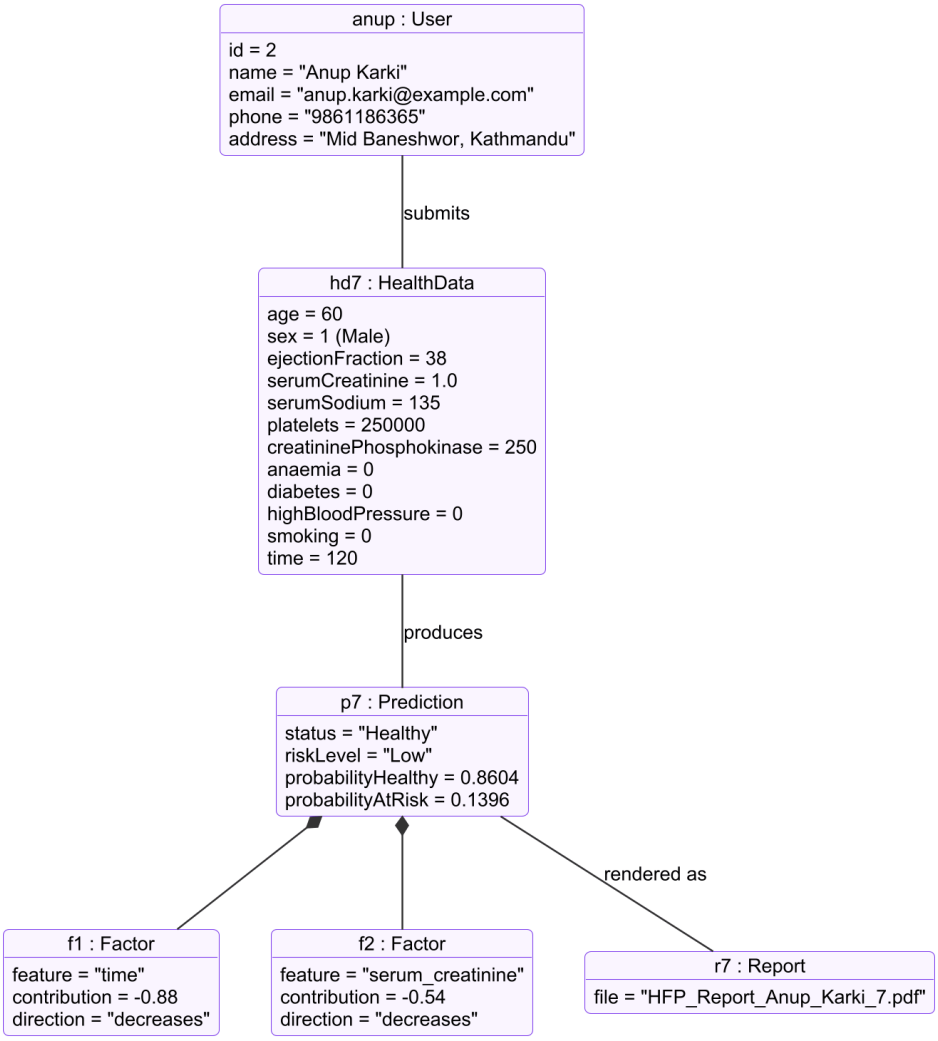


Figure 7: Object Diagram of the HFP System

3.1.4 Dynamic Modelling using State and Sequence Diagrams

The state diagram in Figure 8 models the states a user passes through. A logged-out user can register, log in or reset the password. Inside the logged-in state, the user moves between entering health data, validation, prediction, viewing the result, downloading the report and viewing history. Logging out or an expired token returns the user to the logged-out state.

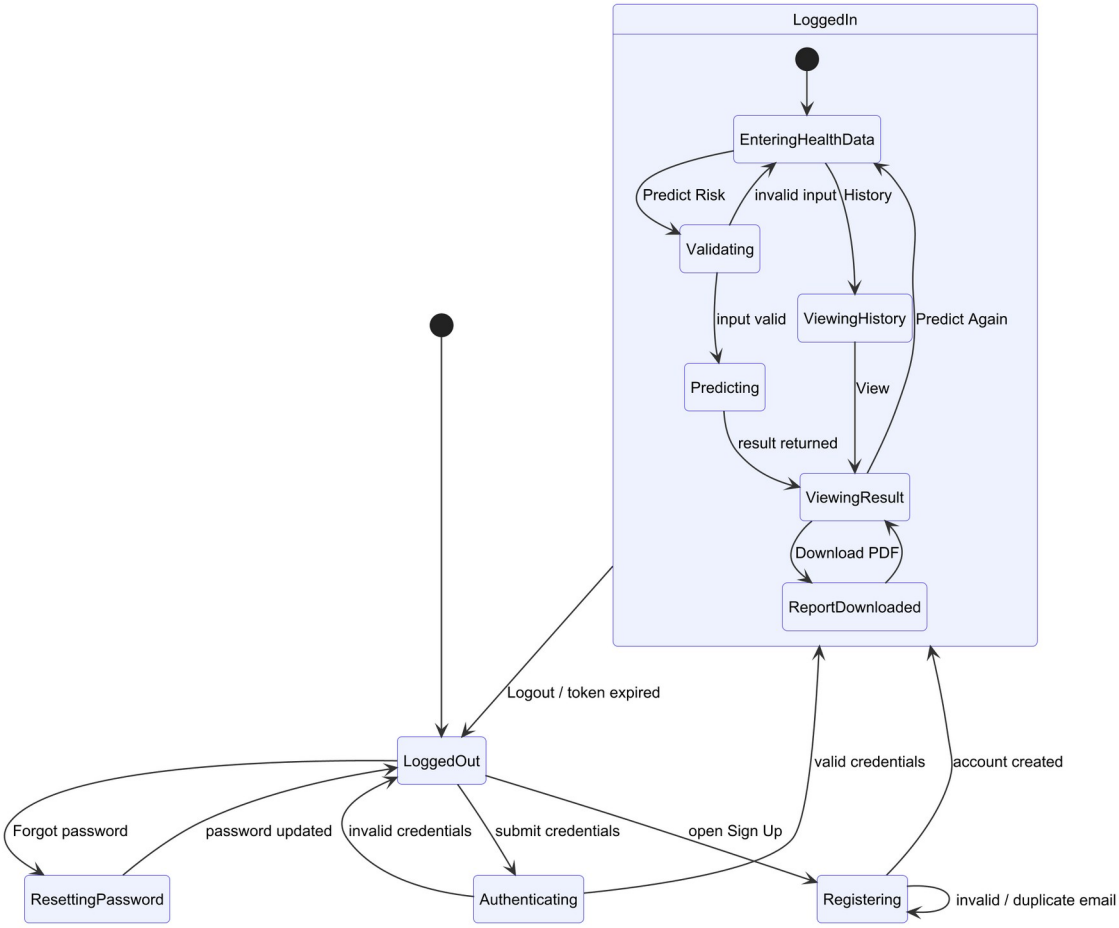


Figure 8: State Diagram of the HFP System

The sequence diagram in Figure 9 shows how the user, frontend, backend, database and ML model interact over time during login, prediction and report download.

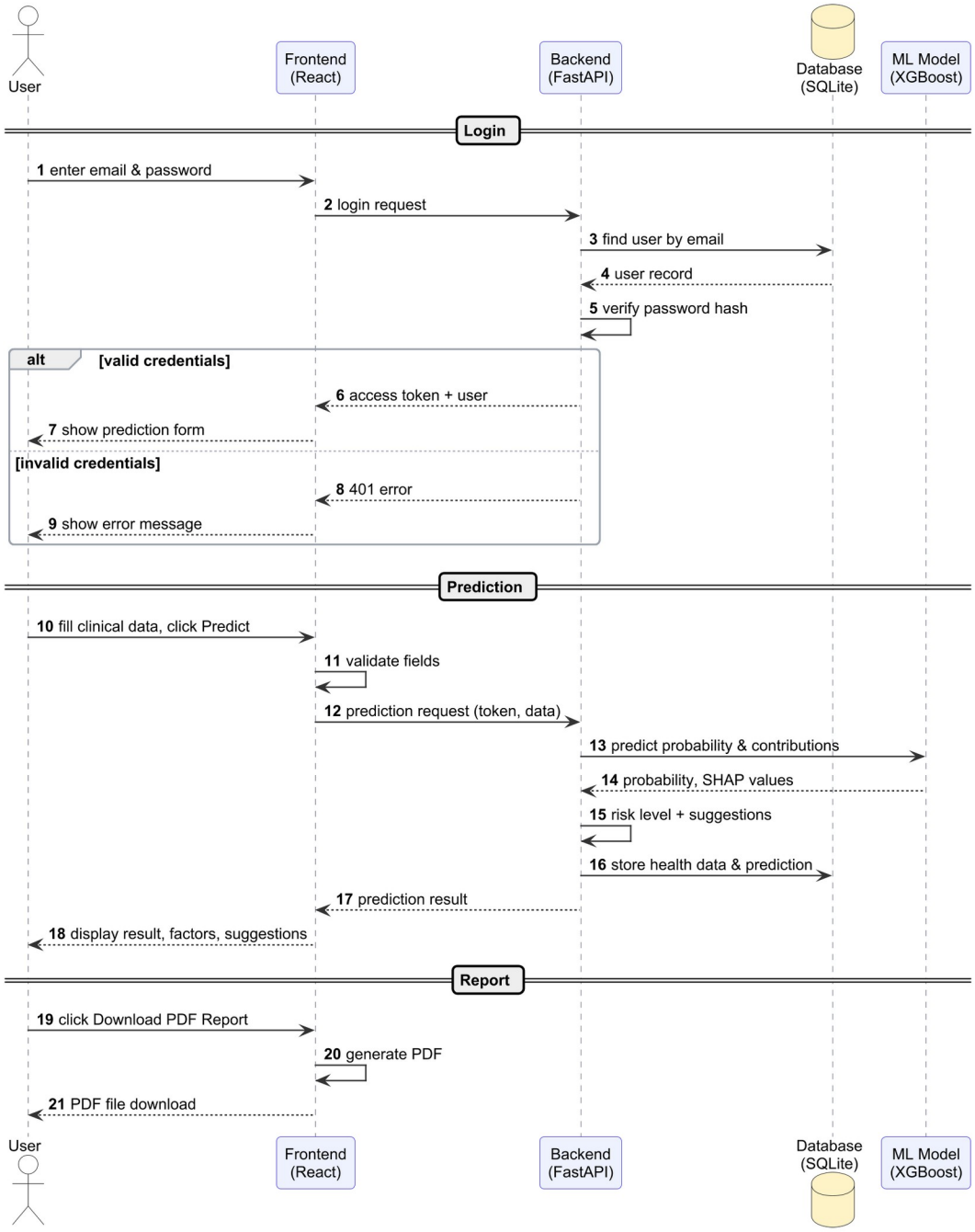


Figure 9: Sequence Diagram of the HFP System

3.1.5 Process Modelling using Activity Diagrams

The activity diagram in Figure 10 describes the complete prediction process from the user's point of view, including the decision points for authentication, input validation, report download and repeated prediction.

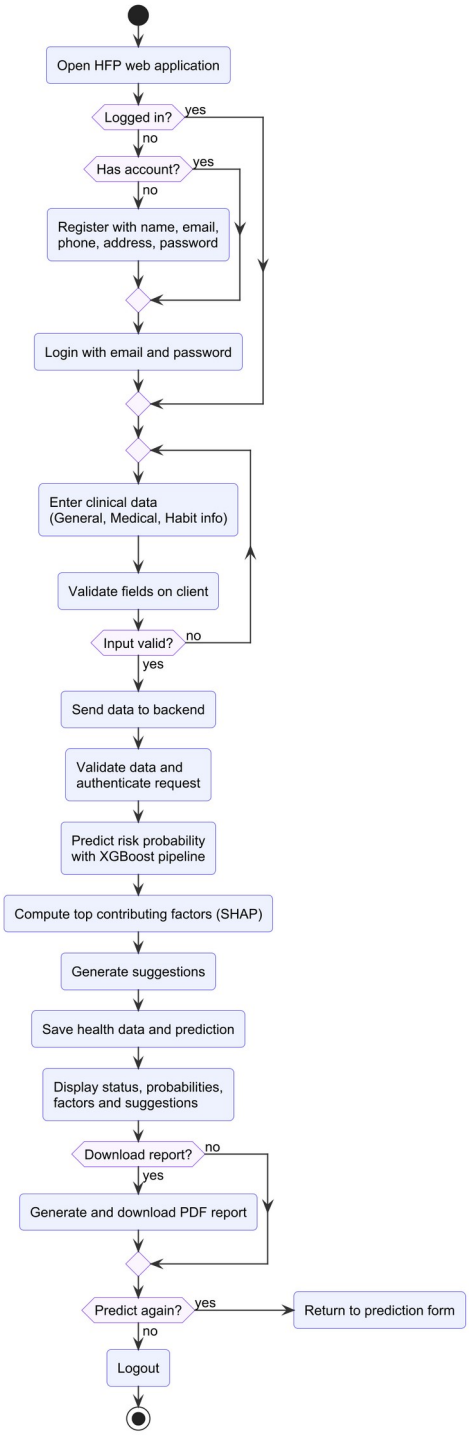


Figure 10: Activity Diagram of the Prediction Process

3.2 System Design

The system design refines the analysis models into the actual structure of the implemented system. Figure 11 gives an overview of the architecture. An offline machine learning pipeline prepares the data, trains and compares the models and exports the

selected pipeline. The web application loads this pipeline in the FastAPI backend, which serves the React frontend and stores data in SQLite.

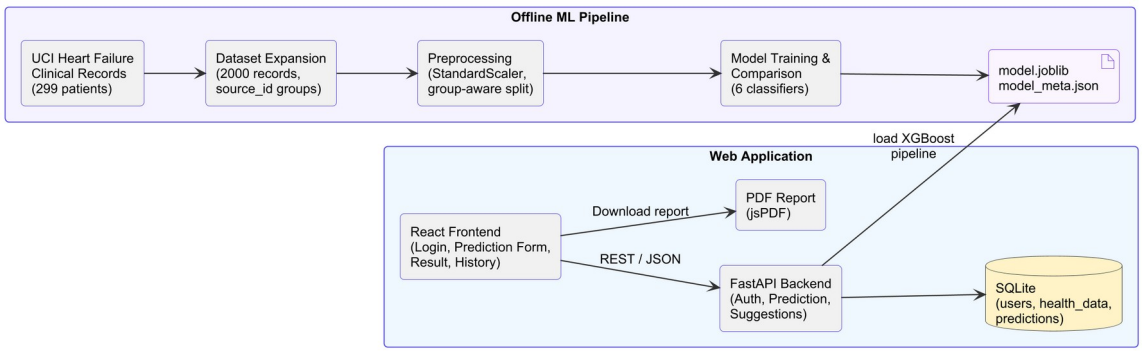


Figure 11: System Architecture (Block Diagram) of the HFP System

3.2.1 Refinement of Class, Object, State, Sequence and Activity Diagrams

Refined class diagram. Figure 12 maps the domain classes to the implemented modules. In the frontend, the *PredictionForm*, *Result* and *History* pages use the *ApiClient* module to call the backend, *AuthProvider* holds the logged-in user, *MedicalInfo* defines the field metadata and validation rules, and *PDFReportGenerator* builds the report. In the backend, *APIRoutes* exposes the REST endpoints and delegates to *Predictor* (model inference and explanations), *Security* (hashing and tokens), *Suggestions* (rule engine) and *Database* (SQLite access). *HealthData* is a Pydantic schema that validates the twelve clinical inputs.

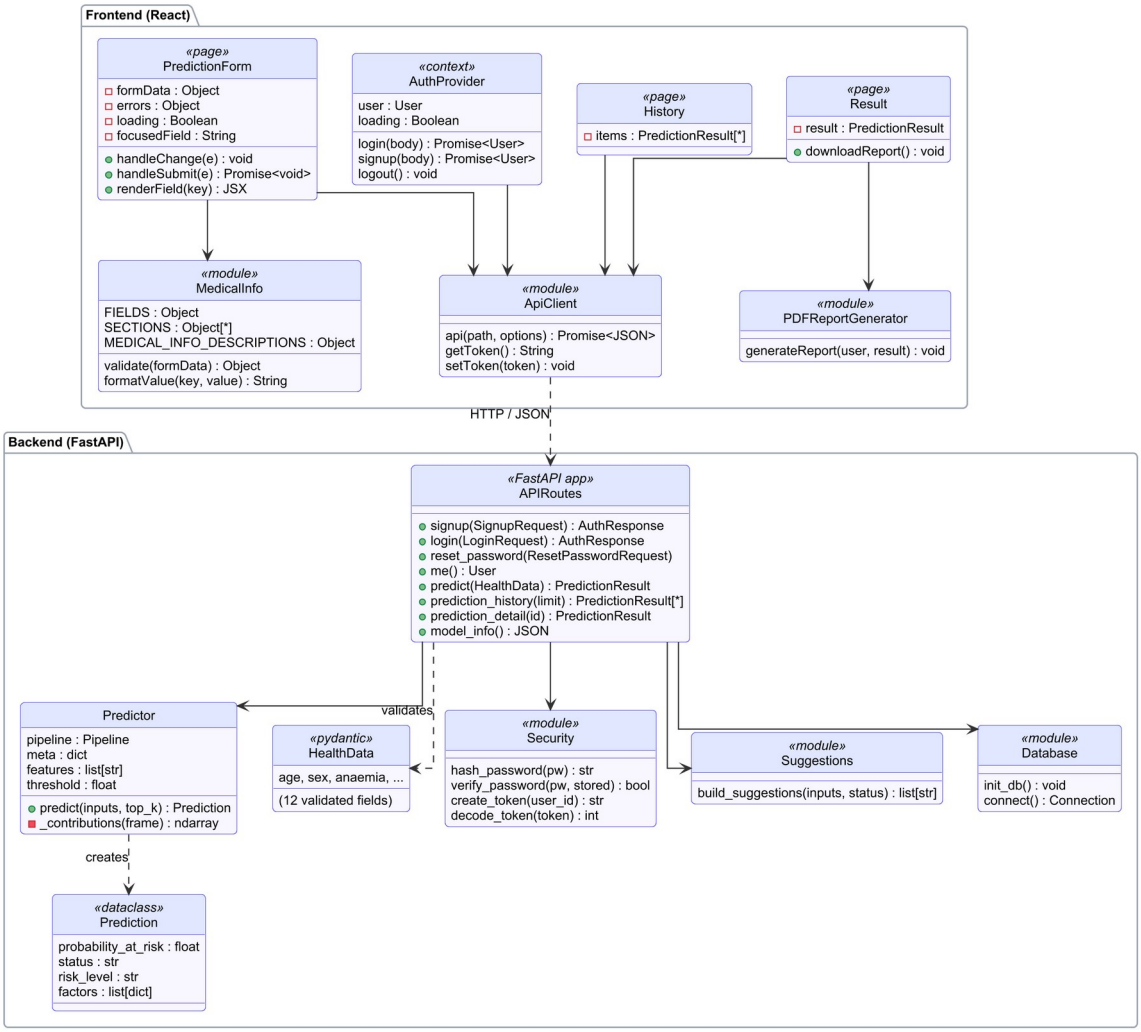


Figure 12: Refined Class Diagram of the HFP System

Refined object diagram. Figure 13 shows the objects that exist in the running backend. The FastAPI application holds a single *Predictor*, loaded once at start-up. The Predictor wraps a scikit-learn *Pipeline* made of a *StandardScaler* and the tuned *XGBClassifier*.

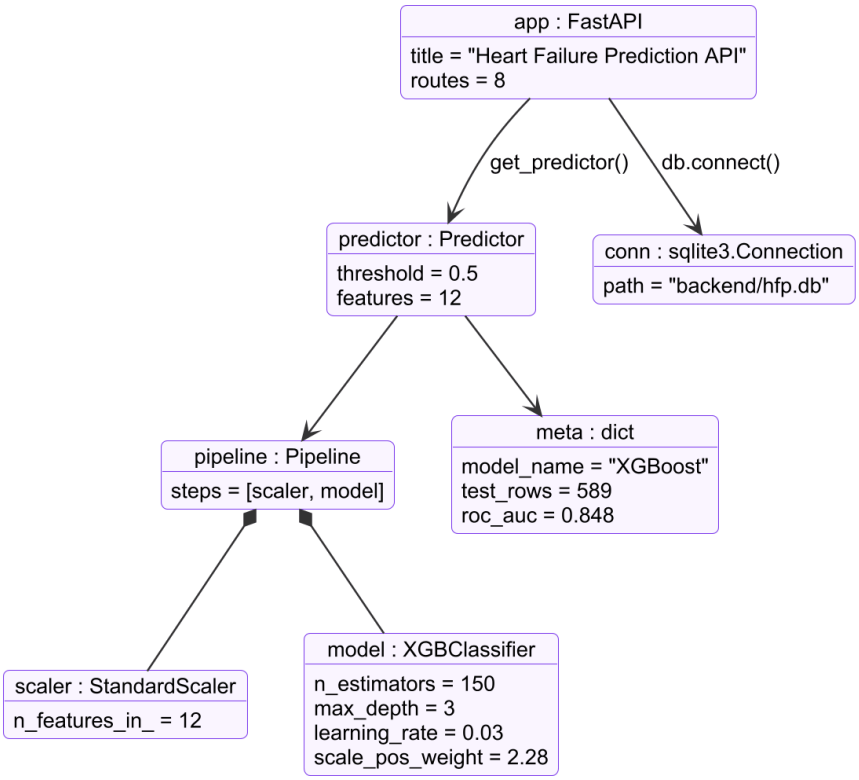


Figure 13: Refined Object Diagram of the Backend at Run Time

Refined state diagram. Figure 14 refines the prediction step into the states of a single *POST /api/predict* request on the server. Each state matches a step of the implementation, and the two failure paths return HTTP 401 (unauthenticated) or 422 (invalid input).

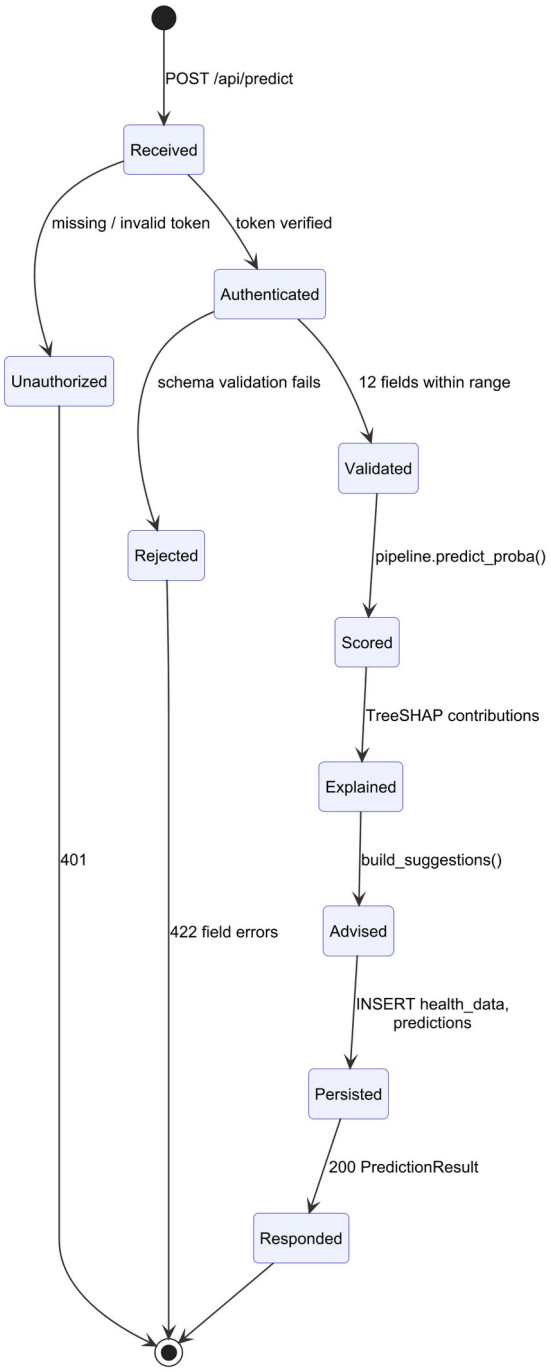


Figure 14: Refined State Diagram of a Prediction Request

Refined sequence diagram. Figure 15 shows the prediction flow with the actual functions and endpoints called at each step, from *handleSubmit* in the form to the downloaded PDF file.

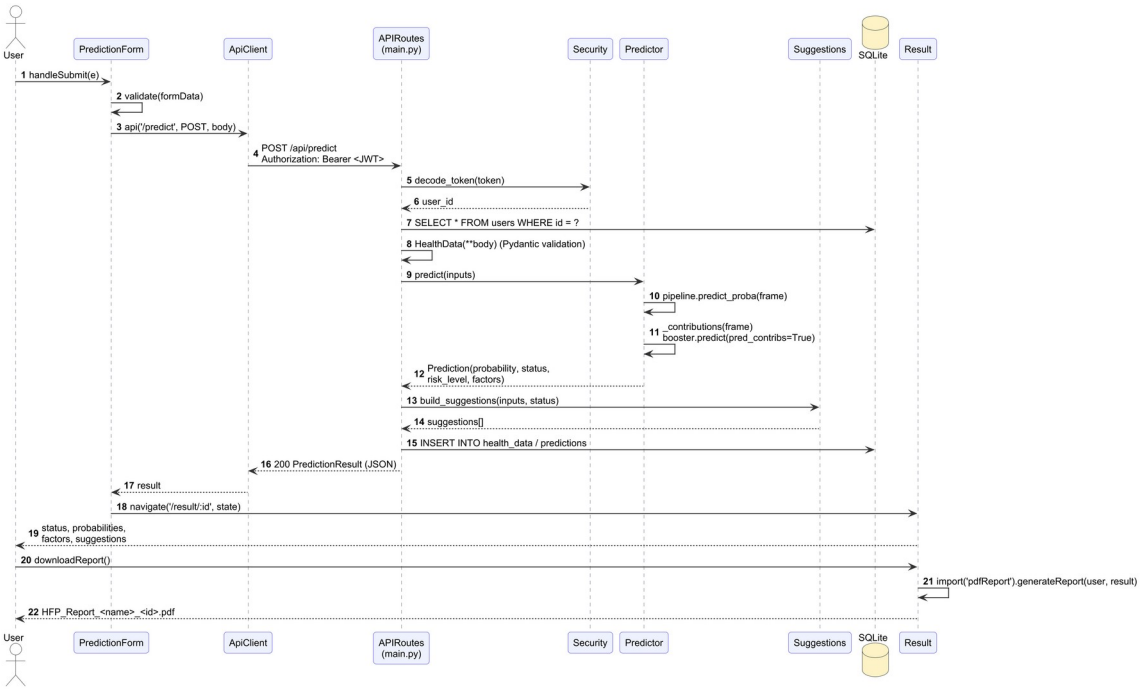


Figure 15: Refined Sequence Diagram of the Prediction Flow

Refined activity diagram. Figure 16 details the offline model training process implemented in *make_dataset.py* and *train.py*. The six models are trained in parallel branches, evaluated in the same way and compared before the XGBoost pipeline is exported.

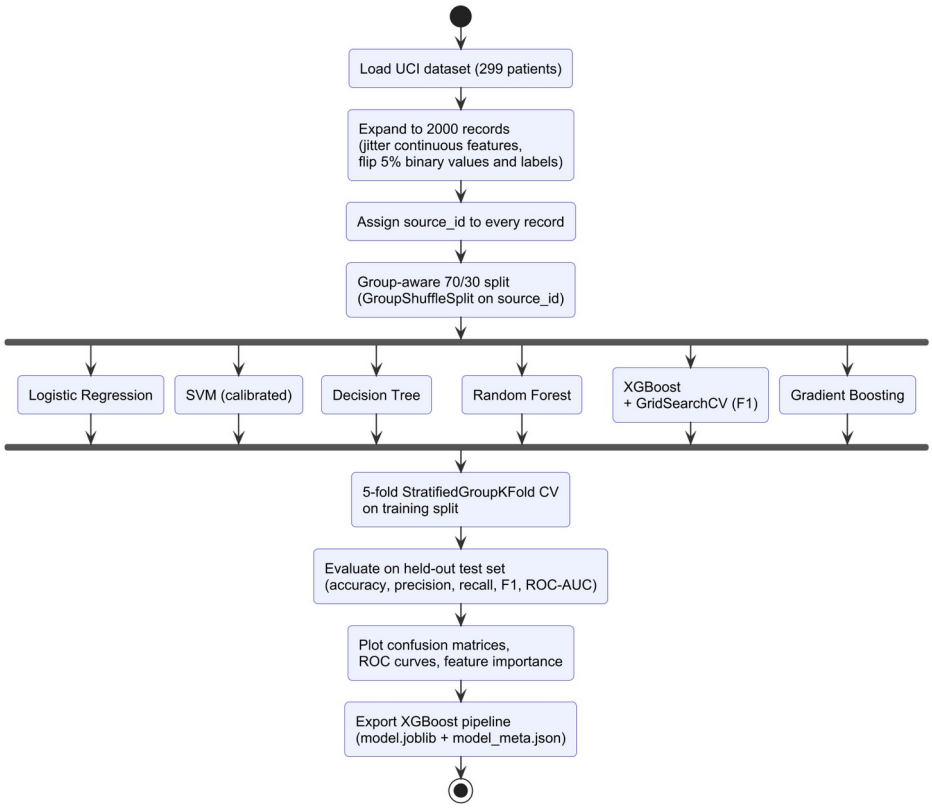


Figure 16: Refined Activity Diagram of the Model Training Pipeline

3.2.2 Component Diagram

The component diagram in Figure 17 shows the software components and their dependencies. The React pages depend on the authentication, API client, PDF and field-metadata components. The API client is the only component that communicates with the server, through the REST API. On the server, the FastAPI application depends on the authentication, prediction, suggestion and persistence modules. The prediction module reads the model artifact produced by the offline training components.

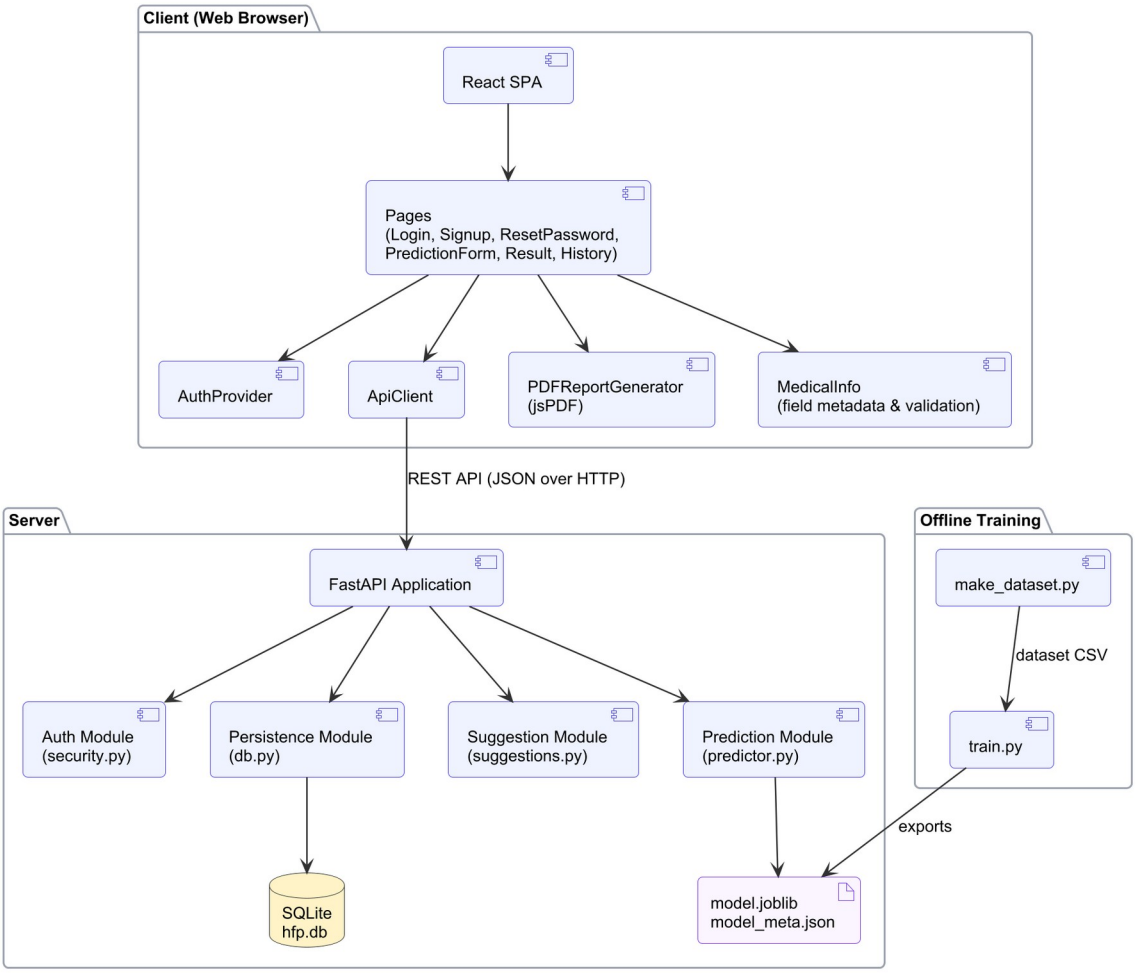


Figure 17: Component Diagram of the HFP System

3.2.3 Deployment Diagram

The deployment diagram in Figure 18 shows how the components are placed on hardware. The user's browser runs the React SPA and keeps the JWT in local storage. The application server runs the FastAPI application on the Uvicorn ASGI server. It also serves the built frontend files, loads the model artifact and stores data in an SQLite file. The model is trained on a developer workstation and copied to the server as an artifact.

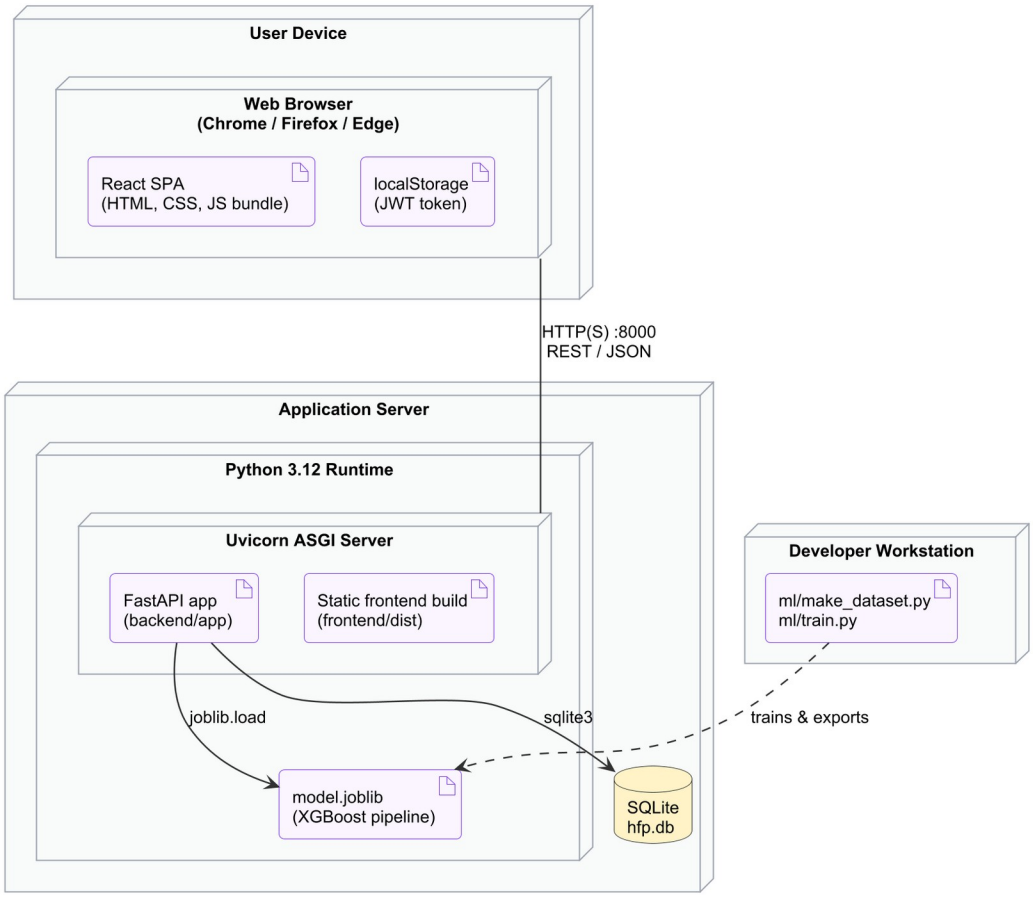


Figure 18: Deployment Diagram of the HFP System

3.3 Algorithm Details

3.3.1 Data Preparation and Preprocessing

Dataset expansion. The 299 real patients were expanded to 2000 records by generating 1701 synthetic patients. Each synthetic patient copies a randomly chosen real patient and then: (i) adds Gaussian noise of 15% of the feature's standard deviation to each continuous feature, applied in log space for the skewed laboratory values (CPK, serum creatinine and platelets); (ii) clips each value to its clinically plausible range and rounds it like the original data; (iii) flips each binary feature with a probability of 5%; and (iv) flips the label with a probability of 5%, so that the expanded dataset is not unrealistically easy to separate. Every record keeps a *source_id* identifying its real patient.

Group-aware split. The data was split 70/30 into training and test sets with *GroupShuffleSplit* on *source_id*, so all synthetic copies of a patient fall on the same side of the split. The training set has 1411 records (209 real patients) and the test set has 589

records (90 real patients). Cross-validation on the training set uses *StratifiedGroupKFold* with five folds for the same reason.

Standardization. Each feature is standardized using the mean and standard deviation of the training set:

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

Class imbalance. The positive class is weighted in Random Forest (balanced class weights) and in XGBoost, where the weight of positive samples is set from the training set:

$$\text{scale_pos_weight} = \frac{\text{number of negative samples}}{\text{number of positive samples}}$$

This gives scale_pos_weight = 981 / 430 = 2.28 for the training split.

3.3.2 Logistic Regression and Support Vector Machine

Logistic Regression models the log-odds of a death event as a linear combination of the standardized features and converts it into a probability with the sigmoid function:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_{12} x_{12}$$

$$P(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The Support Vector Machine uses a radial basis function (RBF) kernel to find a non-linear decision boundary with maximum margin. Because an SVM does not output probabilities directly, its scores were calibrated with *CalibratedClassifierCV*. Both models serve as baselines for the tree-based methods.

3.3.3 Decision Tree Classifier

A decision tree recursively splits the patients on the feature and threshold that best separate the two classes [16]. For example, a split might send *ejection fraction < 30* to one branch and *serum creatinine > 1.5* to another. The quality of a split is measured by the impurity of the resulting subsets, using entropy or the Gini index:

$$\text{Entropy}(S) = -\sum_{i=1}^{c} p_i \log_2 p_i$$

$$\text{Gini}(S) = 1 - \sum_{i=1}^{c} p_i^2$$

The split with the highest information gain is chosen at each node:

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

Splitting stops when a node is pure, the maximum depth is reached, or a node has too few samples. Each leaf predicts its majority class:

$$\text{Leaf} = \arg\max_{c} P(c)$$

To limit overfitting on the small dataset, the tree was restricted to a maximum depth of 5 with at least 10 samples per leaf. Figure 19 illustrates the structure of a decision tree.

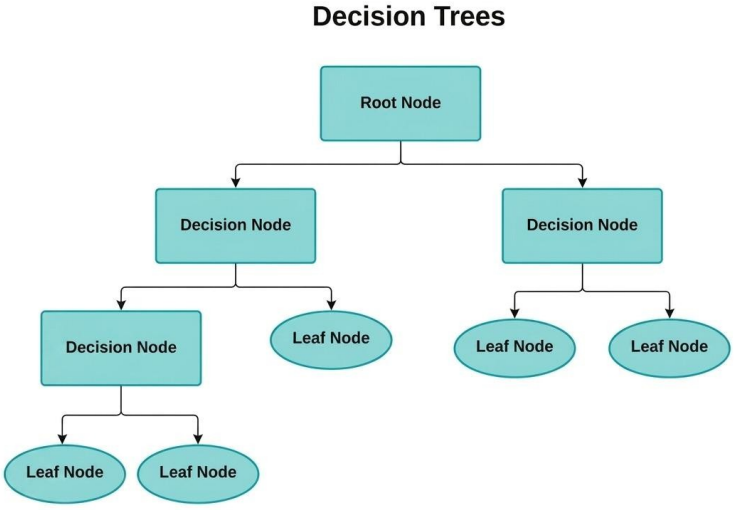


Figure 19: Decision Tree Classifier

3.3.4 Random Forest Classifier

Random Forest builds an ensemble of N decision trees, each trained on a bootstrap sample of the patients and considering a random subset of features at each split [5]. Figure 20 shows this process. The probability of heart failure for patient i is the average of the probabilities given by the trees:

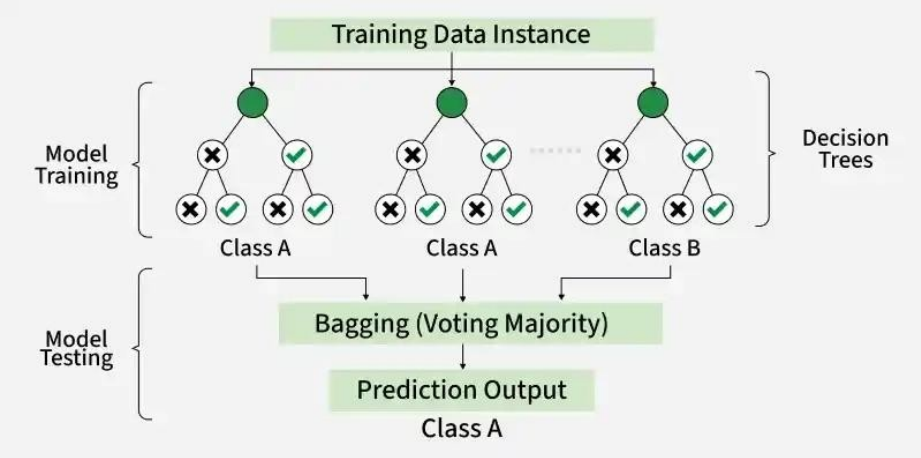


Figure 20: Random Forest Classifier

$$P(\mathrm{HF} \mid X_i) = \frac{1}{N} \sum_{k=1}^{N} p_k(X_i)$$

The final class is obtained with the decision threshold θ:

$$\hat{y}_i = 1 \text{ if } P(\mathrm{HF} \mid X_i) \geq \theta, \text{ otherwise } 0, \text{ with } \theta = 0.5$$

The forest used 300 trees with a maximum depth of 8, at least 5 samples per leaf and balanced class weights.

3.3.5 XGBoost Classifier (Deployed Model)

XGBoost builds trees sequentially, where each new tree f_t corrects the errors of the current ensemble [6], [7]. Figure 21 illustrates this. At iteration t, it minimizes a regularized objective made of the training loss and a complexity penalty:

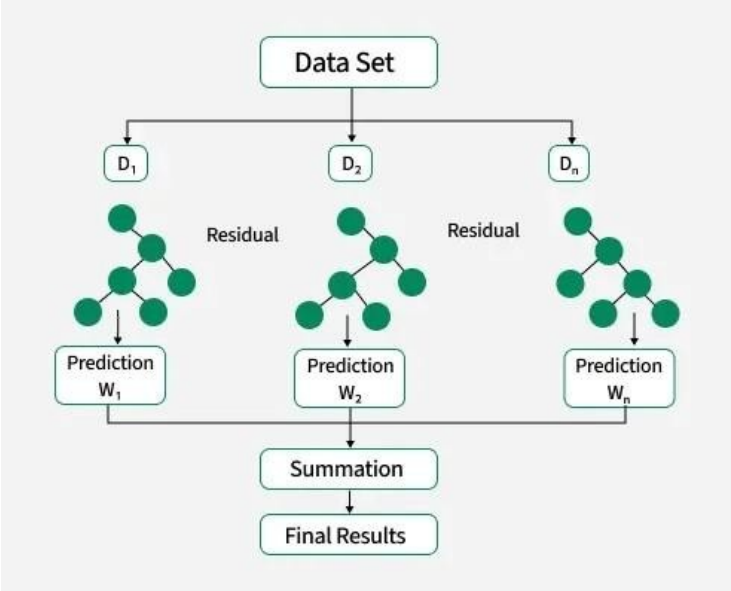


Figure 21: XGBoost Classifier

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l\left(y_i,\ \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t)$$

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

Here T is the number of leaves, w_j the leaf weights, and γ and λ control tree complexity. For binary classification, the loss is the logistic loss:

$$l(y, \hat{p}) = -[\, y \log \hat{p} + (1-y)\log(1-\hat{p})\,]$$

The prediction is updated with a learning rate η, and the final score is converted to a probability with the sigmoid function:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(x_i)$$

Hyperparameter tuning. XGBoost was tuned with a grid search over 24 combinations, scored by F1-score with group-aware five-fold cross-validation on the training set only (Table 11). Subsampling of rows and columns (0.8) and L2 regularization (λ = 1) were fixed.

Table 11: XGBoost Hyperparameter Grid and Selected Values

Hyperparameter	Values searched	Selected
max_depth	2, 3, 4	3
n_estimators	150, 300	150
learning_rate	0.03, 0.1	0.03
min_child_weight	1, 5	1
scale_pos_weight	computed from the training set	2.28

3.3.6 Explaining Predictions with TreeSHAP

For every prediction, the system computes SHAP values φ_j for the twelve features with XGBoost's exact TreeSHAP algorithm [9] (*pred_contribs=True*). The values satisfy the additivity property, where φ_0 is the expected model output (in log-odds) and f(x) is the output for the patient:

$$f(x) = \phi_0 + \sum_{j=1}^{12} \phi_j$$

The features are sorted by |φ_j|, and the five largest are shown to the user. A positive φ_j means the feature *increases* the risk, and a negative φ_j means it *decreases* the risk.

3.3.7 Risk Levels and Suggestion Rules

The risk probability p is converted into a status with a threshold of 0.5 (*At Risk* if $p \geq 0.5$, otherwise *Healthy*), and into one of four risk levels (Table 12). A rule engine then produces suggestions from clinical reference ranges (Table 13).

Table 12: Risk Level Thresholds

Risk probability p	Risk level	Status
$p < 0.25$	Low	Healthy
$0.25 \leq p < 0.50$	Moderate	Healthy
$0.50 \leq p < 0.75$	High	At Risk
$p \geq 0.75$	Very High	At Risk

Table 13: Rules of the Suggestion Engine

Condition	Suggestion given
Ejection fraction < 40%	Reduced pumping function; arrange an echocardiogram review
Serum creatinine > 1.5 mg/dL	Possible reduced kidney function; kidney tests and medication review
Serum sodium < 135 mEq/L	Low sodium; discuss fluid intake and diuretics
CPK > 582 mcg/L	Possible muscle or heart stress; follow-up blood test
Platelets outside 150,000–450,000	Blood work-up advised
High blood pressure = yes	Reduce salt, monitor BP, take medication consistently
Diabetes = yes	Maintain blood sugar control and monitor HbA1c
Anaemia = yes	Iron-rich foods and a haemoglobin check-up
Smoking = yes	Quit smoking
Status = At Risk	Consult a cardiologist soon; watch for warning signs
Status = Healthy	Maintain a heart-healthy lifestyle and routine check-ups

3.3.8 Evaluation Metrics

Models are compared using the confusion matrix counts (TP, TN, FP, FN) on the held-out test set. Because missing a high-risk patient (a false negative) is costly, recall and F1-score are given more weight than accuracy alone:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

The ROC-AUC measures how well the predicted probabilities rank death-event patients above survivors at all thresholds. It is independent of the 0.5 threshold.

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Implementation

The system was implemented in three parts: an offline machine learning pipeline (*ml/*), a REST API backend (*backend/*) and a web frontend (*frontend/*). The source code is managed with Git. Screenshots of the implemented system are given in Appendix A, and selected source code in Appendix B.

4.1.1 Tools Used

Table 14: Tools and Technologies Used

Category	Tool / Technology	Purpose in the Project
Programming languages	Python 3.12, JavaScript (ES2022), SQL	Model, backend; frontend; database queries
Machine learning	scikit-learn 1.9 [17], XGBoost 3.4 [7]	Preprocessing, six classifiers, grid search, metrics, TreeSHAP
Data analysis	pandas 3.0, NumPy 2.4	Loading, expanding and splitting the dataset
Visualization	Matplotlib 3.11, Seaborn 0.13	Confusion matrices, ROC curves, feature importance, charts
Model persistence	joblib 1.6	Saving and loading the trained pipeline
Backend framework	FastAPI 0.142, Uvicorn 0.54, Pydantic 2.13	REST API, ASGI server, input validation
Security	PyJWT 2.15, Python hashlib (PBKDF2)	Access tokens and password hashing
Database platform	SQLite 3	Users, health data and predictions
Frontend	React 19, React Router 7, Vite 8	Single page application, routing, build tool
Reporting	jsPDF 4	Client-side PDF report generation
CASE tools	PlantUML, Graphviz	UML diagrams in Chapter 3
Testing	pytest 9, FastAPI TestClient, headless Google Chrome	Unit, integration and system tests
Development tools	Visual Studio Code, Git, uv, npm	Editing, version control, dependency management

4.1.2 Implementation Details of Modules

a. Dataset Preparation Module (ml/make_dataset.py)

The *expand()* function implements the data expansion described in Section 3.3.1. It assigns a *source_id* to the 299 real patients, samples 1701 base rows with replacement and perturbs them. The noise level, flip probabilities and clinical bounds are module constants, and a fixed random seed (42) makes the generated dataset reproducible. The core of the perturbation is:

```
for col in CONTINUOUS_FEATURES:
    low, high, decimals = BOUNDS[col]
```

```
        if col in LOG_SCALE_FEATURES:
            logged = np.log(df[col])
            noisy = np.exp(np.log(base[col]) + rng.normal(0, NOISE_SCALE *
logged.std(), n_new))
        else:
            noisy = base[col] + rng.normal(0, NOISE_SCALE * df[col].std(),
n_new)
        base[col] = np.clip(noisy, low, high).round(decimals)

    for col in BINARY_FEATURES:
        flip = rng.random(n_new) < BINARY_FLIP_P
        base.loc[flip, col] = 1 - base.loc[flip, col]

    flip = rng.random(n_new) < LABEL_FLIP_P
    base.loc[flip, TARGET] = 1 - base.loc[flip, TARGET]
```

b. Model Training Module (ml/train.py)

build_models() creates the six classifiers, and each one is wrapped in a scikit-learn *Pipeline* with a *StandardScaler*. Scaling is therefore learned only from the training data and applied automatically at prediction time. For each model, *main()* runs group-aware cross-validation, fits the pipeline, computes the test metrics with *evaluate()*, and saves the confusion matrix and ROC curve. XGBoost is tuned with *GridSearchCV*. The selected pipeline is exported to *model.joblib*, together with *model_meta.json*, which contains the feature order, threshold, metrics, feature importances and the full comparison table.

```
        pipeline = Pipeline([("scaler", StandardScaler()), ("model",
estimator)])
        if key == "xgboost":
            search = GridSearchCV(pipeline, XGB_GRID, scoring="f1", cv=cv,
n_jobs=-1)
            search.fit(X_train, y_train, groups=groups[train_idx])
            pipeline = search.best_estimator_
            print(f"XGBoost best params: {search.best_params_}")
        cv_scores = cross_validate(
            pipeline, X_train, y_train, groups=groups[train_idx], cv=cv,
scoring=["accuracy", "precision"]
        )
```

c. Prediction Module (backend/app/predictor.py)

The *Predictor* class loads the pipeline and its metadata once, when the server starts. *predict()* arranges the inputs in the trained feature order, obtains the risk probability, determines the status and risk level, and calls *_contributions()* to compute the SHAP values. For XGBoost, the contributions are obtained directly from the booster:

```
        scaler = self.pipeline.named_steps["scaler"]
        model = self.pipeline.named_steps["model"]
        scaled = scaler.transform(frame)
        if hasattr(model, "get_booster"):
            import xgboost as xgb

            contribs = model.get_booster().predict(xgb.DMatrix(scaled,
feature_names=self.features), pred_contribs=True)
            return contribs[0, :-1]  # last column is the bias term
```

d. Suggestion Module (backend/app/suggestions.py)

build_suggestions() applies the rules of Table 13 to the inputs and the predicted status, and returns an ordered list of plain-language suggestions. Keeping the rules in a separate module allows clinicians to review and change them without touching the model.

e. Authentication Module (backend/app/security.py)

Passwords are hashed with PBKDF2-HMAC-SHA256, using a random 16-byte salt and 310,000 iterations. The hash is stored as *pbkdf2_sha256$iterations$salt$digest*, and *verify_password()* compares digests in constant time. *create_token()* issues an HS256-signed JWT that expires after 12 hours. The FastAPI dependency *current_user()* decodes the token on every protected request and loads the user from the database.

```
def hash_password(password: str) -> str:
    salt = secrets.token_hex(16)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode(),
bytes.fromhex(salt), PBKDF2_ITERATIONS)
    return f"pbkdf2_sha256${PBKDF2_ITERATIONS}${salt}${digest.hex()}"


def verify_password(password: str, stored: str) -> bool:
    try:
        _, iterations, salt, expected = stored.split("$")
    except ValueError:
        return False
    digest = hashlib.pbkdf2_hmac("sha256", password.encode(),
bytes.fromhex(salt), int(iterations))
    return hmac.compare_digest(digest.hex(), expected)
```

f. Persistence Module (backend/app/db.py)

The database schema follows the class diagram. Each prediction request stores one *health_data* row (the validated inputs as JSON) and one *predictions* row (status, probabilities, factors and suggestions). Foreign keys with cascading deletes link the tables (Table 15).

Table 15: Database Schema

Table	Column	Type	Description
users	id	INTEGER PK	Auto-increment identifier
	name, email, phone, address	TEXT	Profile; email is UNIQUE
	password_hash	TEXT	PBKDF2 hash with salt
	created_at	TEXT	Creation timestamp
health_data	id	INTEGER PK	Auto-increment identifier
	user_id	INTEGER FK	References users(id)
	inputs_json	TEXT	Twelve validated clinical inputs
predictions	id	INTEGER PK	Auto-increment identifier
	health_data_id	INTEGER FK	References health_data(id)
	status, risk_level	TEXT	Healthy / At Risk; Low – Very High
	probability_healthy,	REAL	Predicted probabilities

Table	Column	Type	Description
	probability_at_risk		
	details_json	TEXT	Top factors and suggestions

g. API Module (backend/app/main.py)

The FastAPI application defines the REST endpoints in Table 16. Request and response bodies are Pydantic models. Out-of-range values (for example, an ejection fraction outside 5–90%) are rejected with HTTP 422 and a message for each field, which the frontend shows next to the corresponding input. The application also serves the built frontend, so the whole system runs as a single process.

Table 16: REST API Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/signup	No	Register a user; returns token and user
POST	/api/auth/login	No	Authenticate; returns token and user
POST	/api/auth/reset-password	No	Reset password using email and phone
GET	/api/auth/me	Yes	Current user profile
POST	/api/predict	Yes	Predict risk, explain, suggest and store
GET	/api/predictions	Yes	The user's prediction history
GET	/api/predictions/{id}	Yes	One prediction of the user
GET	/api/model	No	Model name, metrics and feature importances

h. Frontend Modules (frontend/src)

PredictionForm keeps the form state (*formData*, *errors*, *loading*, *focusedField*), renders the three input sections from the *SECTIONS* metadata, shows the description of the focused field, and validates the values with the same ranges as the backend before sending them. *Result* displays the status card, probability bar, factor bars and suggestions, and lazily loads *PDFReportGenerator* (jsPDF) only when the report is downloaded. *History* lists past predictions. *AuthProvider* restores the session from the stored token on page load and logs the user out automatically when the server returns HTTP 401.

4.2 Testing

Testing was carried out at three levels. Automated unit tests check individual functions in isolation. Automated integration tests exercise the API end-to-end with an isolated temporary database. System tests run the complete application in a real browser (headless Google Chrome). All 19 automated tests and all 14 system test cases passed.

4.2.1 Test Cases for Unit Testing

Table 17 lists the unit and integration test cases, implemented with pytest in *backend/tests* and *ml/tests*.

Table 17: Unit and Integration Test Cases

ID	Module	Test Case	Input	Expected Result	Result
UT-01	Dataset	Expanded dataset is valid	UCI data, 1000 rows	No missing values; binary columns in {0,1}; values within bounds	Pass
UT-02	Dataset	Expansion is reproducible	Same seed twice	Identical data frames	Pass
UT-03	Security	Password hash round trip	"supersecret1"	Correct password verifies; wrong one fails	Pass
UT-04	Security	Hashes are salted	Same password twice	Different hashes	Pass
UT-05	Security	Malformed hash rejected	"not-a-valid-hash"	Verification returns false	Pass
UT-06	Security	Token round trip and tampering	Token for user 42; altered token	Returns 42; tampered token rejected	Pass
UT-07	Prediction	Risk level boundaries	p = 0.10, 0.25, 0.50, 0.75	Low, Moderate, High, Very High	Pass
UT-08	Prediction	Predictor output	High-risk patient	p in [0,1], At Risk, 5 factors sorted by |contribution|	Pass
UT-09	Suggestions	Abnormal values	EF 20, creatinine 3.5, sodium 128, BP, smoker	Suggestions for each abnormality and cardiologist advice	Pass
UT-10	Suggestions	Normal values	Healthy patient	Only the healthy-lifestyle suggestion	Pass
IT-01	Auth API	Sign up, login and profile	Valid user data	201 Created; login works case-insensitively; profile returned	Pass
IT-02	Auth API	Duplicate registration	Same email twice	409 Conflict	Pass
IT-03	Auth API	Wrong password	Wrong password	401 Unauthorized	Pass
IT-04	Auth API	Signup validation	Short password, invalid phone	422 with field errors	Pass
IT-05	Auth API	Reset password	Wrong phone, then correct phone	400; then 200 and login with new password	Pass
IT-06	Predict API	Authentication required	No token; invalid token	401 Unauthorized	Pass
IT-07	Predict API	Healthy vs high-risk patient	Two patient profiles	Healthy and At Risk; probabilities sum to 1	Pass
IT-08	Predict API	Input validation	EF = 150, smoking = 2, missing age	422 naming the invalid fields	Pass
IT-09	History API	Per-user isolation	Two users	Own history only; other user's record gives 404	Pass
IT-10	Model API	Model metadata	GET /api/model	XGBoost, 12 features, 6 compared models	Pass

4.2.2 Test Cases for System Testing

System test cases (Table 18) were executed on the complete running system (FastAPI serving the production build of the frontend) by an automated browser script that acts like a user. The actual results recorded by the script are reported.

Table 18: System Test Cases

ID	Test Scenario	Expected Result	Actual Result	Status
ST-01	Protected page without login	Redirect to login page	Redirected to /login	Pass

ID	Test Scenario	Expected Result	Actual Result	Status
ST-02	Register new account	Account created and prediction form shown	Account created, prediction form shown	Pass
ST-03	Logout	Login page shown	Returned to login page	Pass
ST-04	Register with existing email	Duplicate email error	Error: "An account with this email already exists"	Pass
ST-05	Login with wrong password	Invalid credentials error	Error: "Invalid email or password"	Pass
ST-06	Reset password	Success message on login page	Success: "Password updated successfully. You can now log in."	Pass
ST-07	Login with new password	Login succeeds	Logged in, prediction form shown	Pass
ST-08	Submit out-of-range values	Submission blocked with field messages	Blocked; messages: Required; Must be between 5 and 90	Pass
ST-09	Predict low-risk patient	Healthy status with probabilities	Status: Healthy Low risk; Risk Probability: 13.96% (40 ms)	Pass
ST-10	Download PDF report	PDF file downloaded	Downloaded HFP_Report_Dipesh_Kumar_Sharma_6.pdf (10158 bytes)	Pass
ST-11	Predict high-risk patient	At Risk status with suggestions	Status: At Risk Very High risk; 7 suggestions shown	Pass
ST-12	View prediction history	Both predictions listed; past result opens	2 records listed; older result reopened	Pass
ST-13	Session persists after reload	User remains logged in	User still logged in after refresh	Pass
ST-14	Mobile layout (390 px)	Responsive layout	Sections stack vertically, no horizontal scroll	Pass

4.3 Result Analysis

4.3.1 Model Comparison

All six models were trained on the same 1411 training records and evaluated on the same 589 held-out records from 90 patients never seen during training. Table 19 and Figure 22 summarize the results.

Table 19: Performance Comparison of the Machine Learning Models

Model	CV Accuracy	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.799	0.764	0.805	0.532	0.641	0.805
SVM	0.785	0.727	0.707	0.528	0.604	0.769
Decision Tree	0.781	0.744	0.697	0.622	0.658	0.734
Random Forest	0.837	0.769	0.724	0.674	0.698	0.848
XGBoost	0.826	0.750	0.699	0.648	0.673	0.848
Gradient Boosting	0.820	0.691	0.647	0.481	0.552	0.775

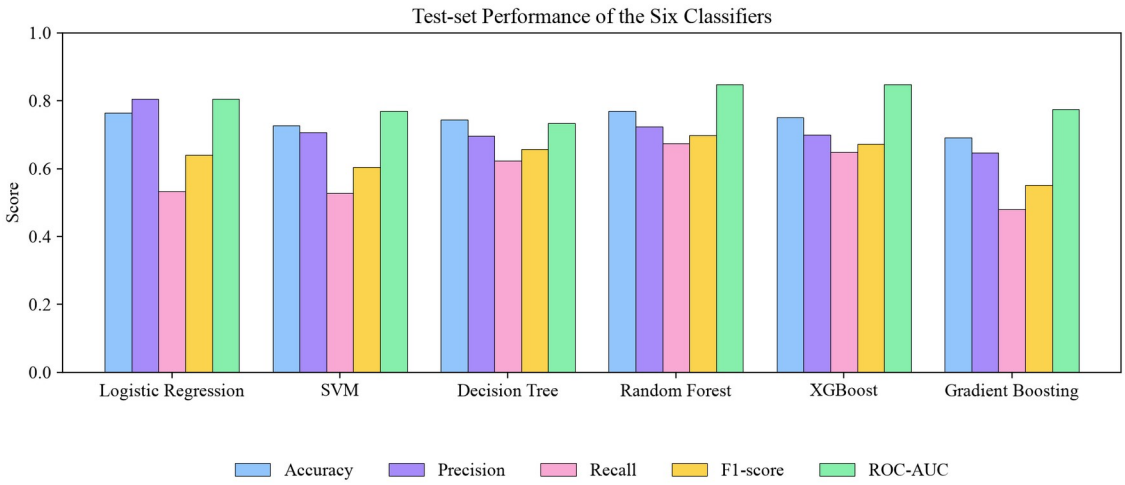


Figure 22: Test-set Performance of the Six Classifiers

Logistic Regression achieved the highest precision (0.805) but a low recall (0.532): it missed almost half of the patients who died, which is unacceptable for a screening tool. Gradient Boosting and SVM showed the same pattern. The tree ensembles gave the best balance. Random Forest obtained the highest accuracy (0.769), recall (0.674) and F1-score (0.698). XGBoost was second on F1-score (0.673) and achieved the highest ROC-AUC (0.8484, against 0.8481 for Random Forest).

XGBoost was selected as the deployed model, as planned in the proposal, for three reasons. First, it discriminates between high- and low-risk patients as well as any model (the highest ROC-AUC), and the F1-score gap to Random Forest is small (0.025). Second, XGBoost computes exact per-patient TreeSHAP explanations natively, which is central to the system's explainability objective. Third, its regularized objective and tuned shallow trees (depth 3) reduce the risk of overfitting the small source dataset. The training pipeline supports deploying Random Forest instead with a single option if higher recall is preferred.

The confusion matrices in Figure 23 show the trade-off in absolute numbers. Of the 233 test records with a death event, XGBoost identified 151 and Random Forest 157, while Logistic Regression identified only 124. Figure 24 compares the ROC curves.

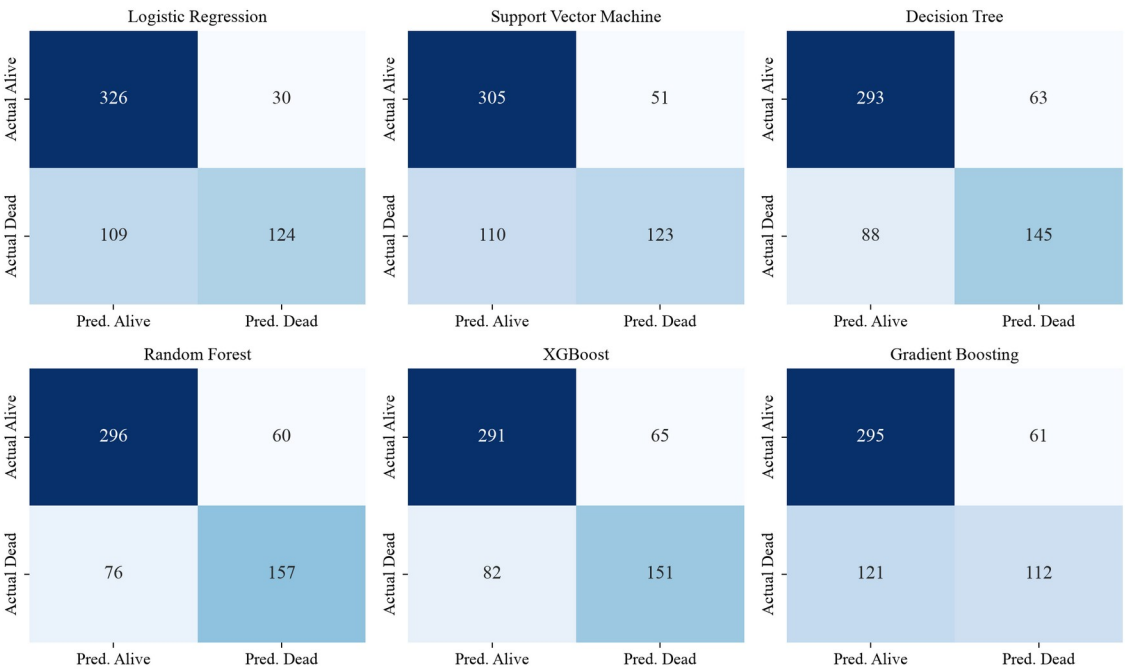


Figure 23: Confusion Matrices of the Six Models on the Test Set

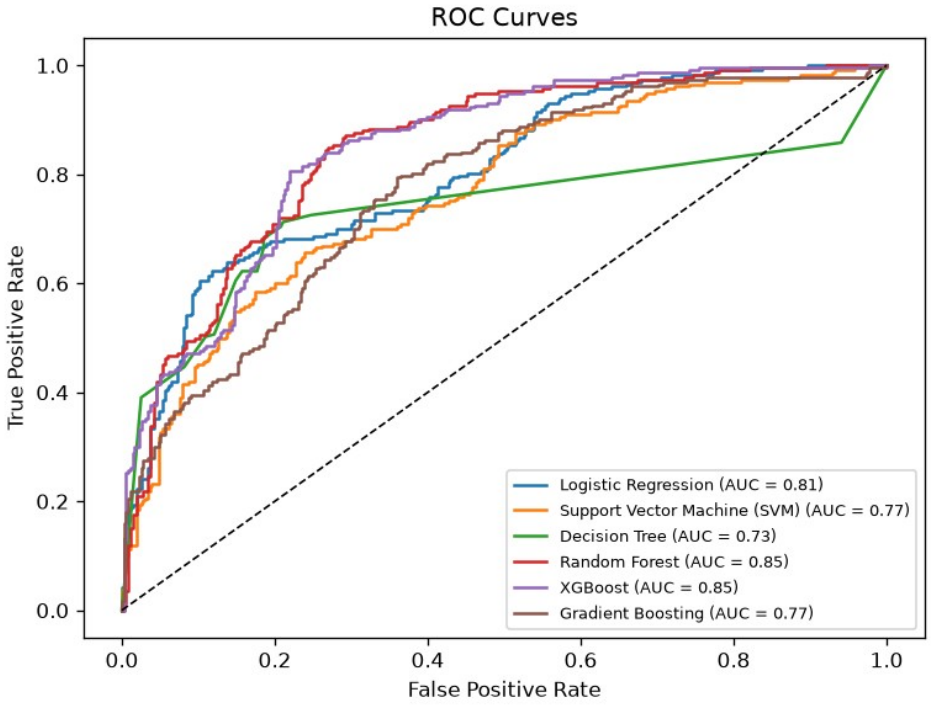


Figure 24: ROC Curves of the Six Models

4.3.2 Feature Importance

Figure 25 shows the global feature importance of the deployed XGBoost model. Follow-up time is the most important feature (32.3%), followed by serum creatinine (13.5%) and ejection fraction (9.6%). Serum creatinine and ejection fraction being the strongest

clinical predictors agrees with the findings of Chicco and Jurman on the same data [3]. The dominance of follow-up time is expected: patients who died early had short follow-up periods. This is why follow-up time is discussed as a limitation in Section 1.4.2.

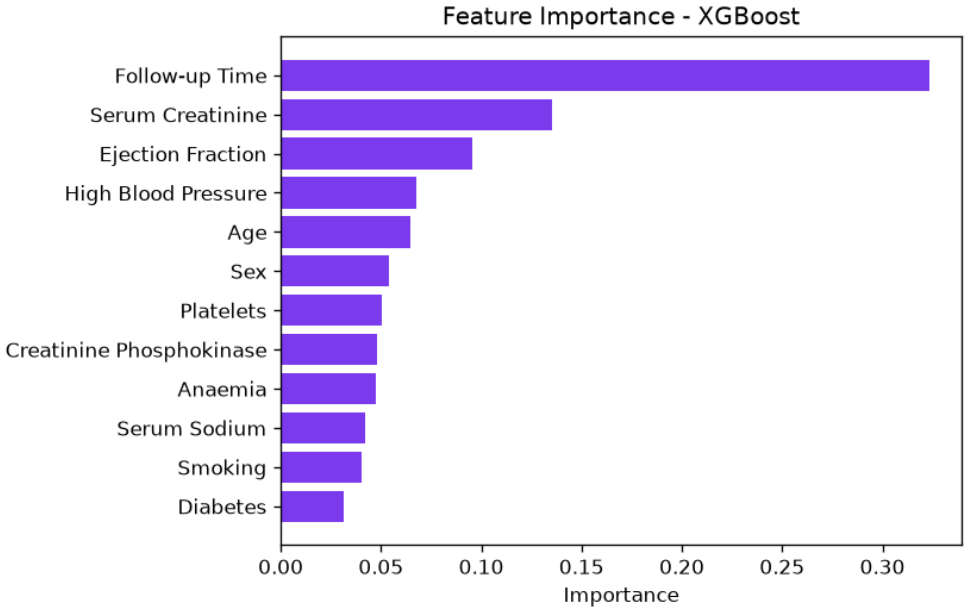


Figure 25: Feature Importance of the XGBoost Model

4.3.3 System Output and Performance

Table 20 shows predictions made by the deployed system for three patient profiles during testing. The outputs are consistent with clinical expectation. The patient with a normal ejection fraction and creatinine is classified as low risk, while the patients with a low ejection fraction, high creatinine, low sodium and short follow-up are classified as very high risk. Their top contributing factors are follow-up time, serum creatinine and ejection fraction.

Table 20: Sample Predictions of the Deployed System

Profile (age, EF, creatinine, sodium, follow-up, other)	Status	Risk level	Risk prob.
60 y, EF 38%, 1.0 mg/dL, 135 mEq/L, 120 days, no comorbidities	Healthy	Low	13.96%
75 y, EF 25%, 2.4 mg/dL, 130 mEq/L, 30 days, high BP, anaemia, CPK 720	At Risk	Very High	91.75%
80 y, EF 20%, 3.0 mg/dL, 128 mEq/L, 15 days, high BP, smoker	At Risk	Very High	97.34%

Response time was measured over 200 consecutive prediction requests to the running server on a development laptop. The mean server round-trip time was 2.2 ms (median 1.9 ms, 95th percentile 2.4 ms), including authentication, model inference, SHAP computation and two database writes. A login took about 22 ms, most of which is the

deliberately slow PBKDF2 password hashing. Both are far below the one-second target. The screens of the final system are shown in Appendix A.

CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Conclusion

This project developed the Heart Failure Prediction (HFP) system, a complete web application that predicts a patient's risk of a heart-failure death event from twelve routine clinical attributes. Six machine learning algorithms were compared using a leakage-free, patient-group-aware evaluation. The deployed XGBoost model achieved a ROC-AUC of 0.848, an accuracy of 75.0% and a recall of 64.8% on held-out patients.

Beyond the model, the system meets all of its objectives. It explains every prediction through SHAP contributions, gives rule-based health suggestions, generates downloadable PDF reports and keeps a secure, per-user prediction history. It was designed using UML models and implemented with FastAPI, React and SQLite. Its correctness was confirmed by 19 automated tests and 14 system test cases. The project shows how routinely collected clinical data and explainable machine learning can be combined into a practical tool that supports early identification of high-risk heart failure patients.

5.2 Future Recommendations

- **Local and larger datasets:** retrain and validate the model on real patient records from Nepali hospitals, replacing the synthetic expansion with genuinely new patients.
- **Additional clinical features:** include ECG findings, natriuretic peptides (BNP/NT-proBNP), medication and admission history, and consider removing follow-up time for a purely prospective model.
- **Probability calibration and threshold tuning:** calibrate the predicted probabilities and choose the decision threshold together with clinicians to favour higher recall.
- **EHR integration:** connect the system to hospital electronic health record systems so that clinical values are filled in automatically.
- **Role-based access:** add clinician and administrator roles, so that a clinician can manage several patients, and add email or SMS one-time-password verification for password reset.

- **Time-series monitoring:** use repeated measurements to track how a patient's risk changes over time.
- **Mobile application and alerts:** provide a mobile app for data entry and notify patients and clinicians when the risk increases.

REFERENCES

[1] G. Savarese, P. M. Becher, L. H. Lund, P. Seferovic, G. M. C. Rosano, and A. J. S. Coats, "Global burden of heart failure: a comprehensive and updated review of epidemiology," *Cardiovascular Research*, vol. 118, no. 17, pp. 3272–3287, 2022, doi: 10.1093/cvr/cvac013.

[2] T. Ahmad, A. Munir, S. H. Bhatti, M. Aftab, and M. A. Raza, "Survival analysis of heart failure patients: A case study," *PLOS ONE*, vol. 12, no. 7, Art. no. e0181001, Jul. 2017, doi: 10.1371/journal.pone.0181001.

[3] D. Chicco and G. Jurman, "Machine learning can predict survival of patients with heart failure from serum creatinine and ejection fraction alone," *BMC Medical Informatics and Decision Making*, vol. 20, no. 1, Art. no. 16, Feb. 2020, doi: 10.1186/s12911-020-1023-5.

[4] "Heart Failure Clinical Records," UCI Machine Learning Repository, 2020. [Online]. Available: https://archive.ics.uci.edu/dataset/519, doi: 10.24432/C5Z89R.

[5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/A:1010933404324.

[6] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, Oct. 2001, doi: 10.1214/aos/1013203451.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD '16)*, San Francisco, CA, USA, Aug. 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[8] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, Long Beach, CA, USA, 2017, pp. 4765–4774.

[9] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, Jan. 2020, doi: 10.1038/s42256-019-0138-9.

[10] R. Detrano, A. Janosi, W. Steinbrunn, M. Pfisterer, J.-J. Schmid, S. Sandhu, K. H. Guppy, S. Lee, and V. Froelicher, "International application of a new probability algorithm for the diagnosis of coronary artery disease," *The American Journal of Cardiology*, vol. 64, no. 5, pp. 304–310, Aug. 1989, doi: 10.1016/0002-9149(89)90524-9.

[11] M. Gudadhe, K. Wankhade, and S. Dongre, "Decision support system for heart disease based on support vector machine and artificial neural network," in *Proc. Int. Conf. Computer and Communication Technology (ICCCT)*, Allahabad, India, Sep. 2010, pp. 741–745, doi: 10.1109/ICCCT.2010.5640377.

[12] M. A. Jabbar, B. L. Deekshatulu, and P. Chandra, "Heart disease prediction system using associative classification and genetic algorithm," in *Proc. Int. Conf. Emerging Trends in Electrical, Electronics and Communication Technologies (ICECIT)*, vol. 1, Elsevier, Dec. 2012, pp. 183–192. [Online]. Available: arXiv:1303.5919.

[13] S. Mohan, C. Thirumalai, and G. Srivastava, "Effective heart disease prediction using hybrid machine learning techniques," *IEEE Access*, vol. 7, pp. 81542–81554, 2019, doi: 10.1109/ACCESS.2019.2923707.

[14] S. K. Lakshmanaprabu, S. N. Mohanty, K. Shankar, N. Arunkumar, and G. Ramirez, "Optimal deep learning model for classification of lung cancer on CT images," *Future Generation Computer Systems*, vol. 92, pp. 374–382, Mar. 2019, doi: 10.1016/j.future.2018.10.009.

[15] A. M. Alaa and M. van der Schaar, "Forecasting individualized disease trajectories using interpretable deep learning," arXiv preprint arXiv:1810.10489, Oct. 2018.

[16] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, Mar. 1986, doi: 10.1007/BF00116251.

[17] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

BIBLIOGRAPHY

FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com

React Documentation. [Online]. Available: https://react.dev

XGBoost Documentation. [Online]. Available: https://xgboost.readthedocs.io

Scikit-learn User Guide. [Online]. Available: https://scikit-learn.org/stable/user_guide.html

PlantUML Language Reference Guide. [Online]. Available: https://plantuml.com

jsPDF Documentation. [Online]. Available: https://github.com/parallax/jsPDF

APPENDICES

Appendix A: Screenshots

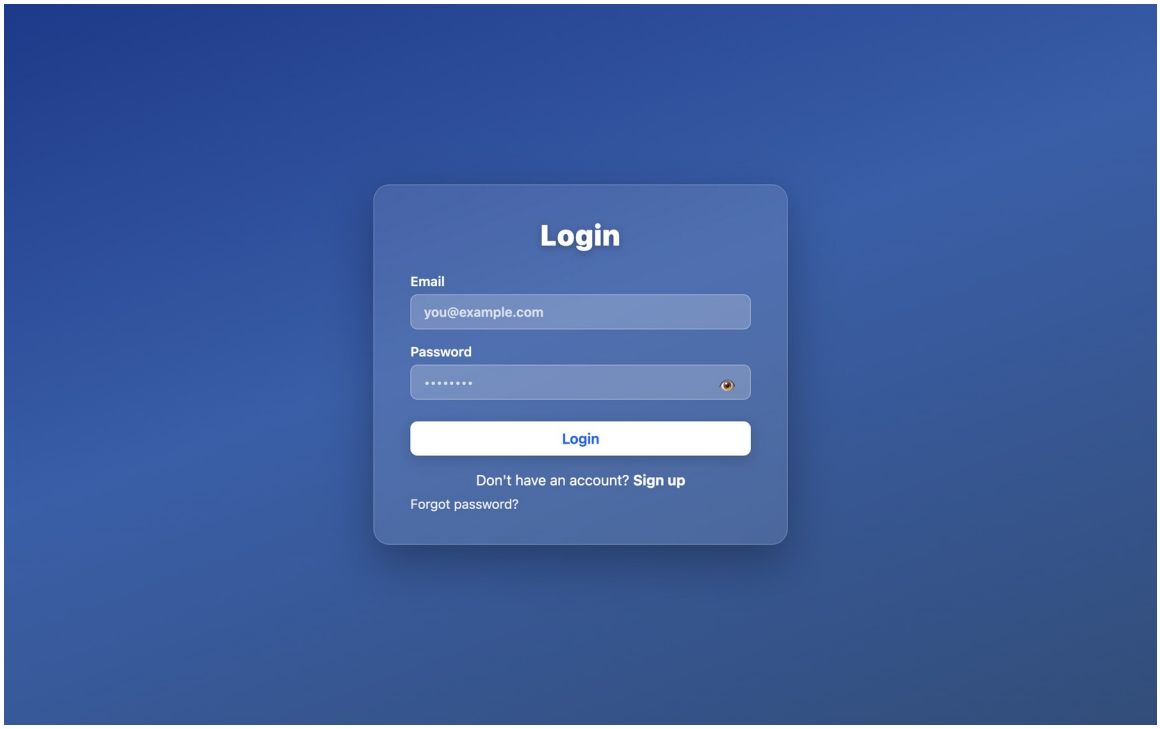


Figure 26: Login Page

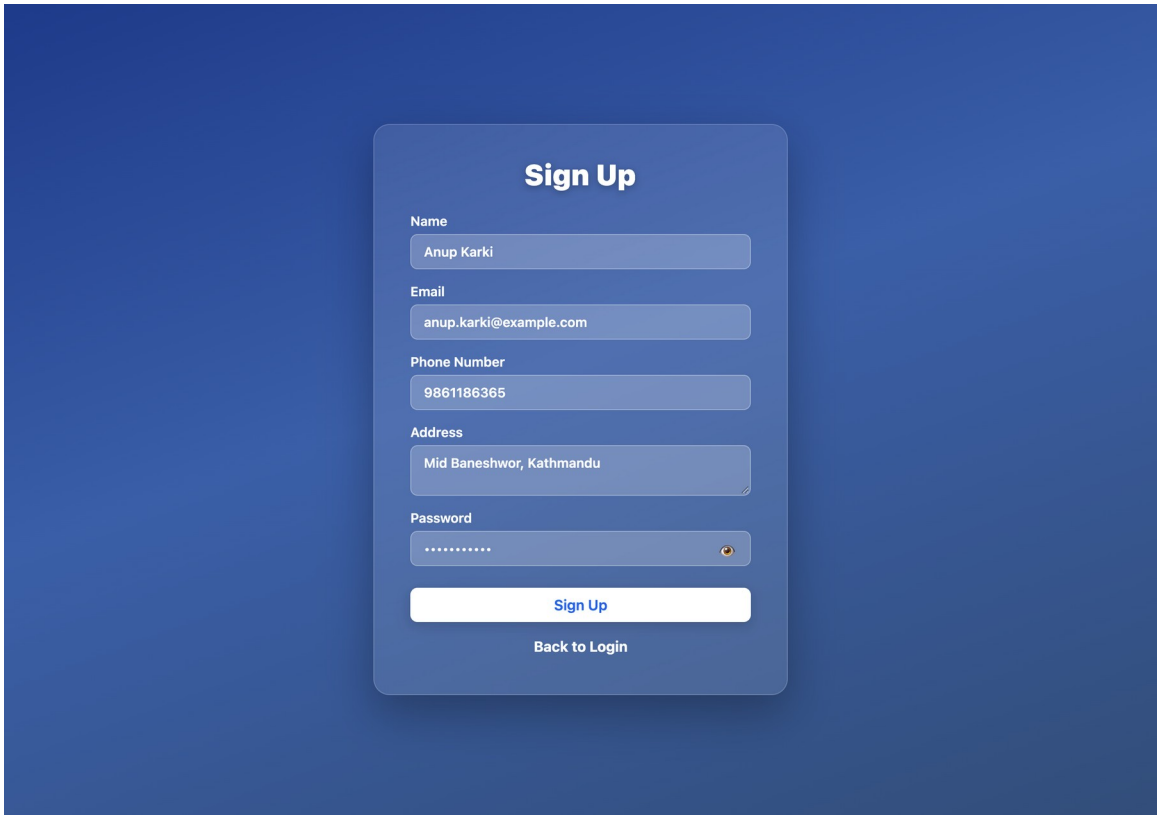


Figure 27: Sign Up Page

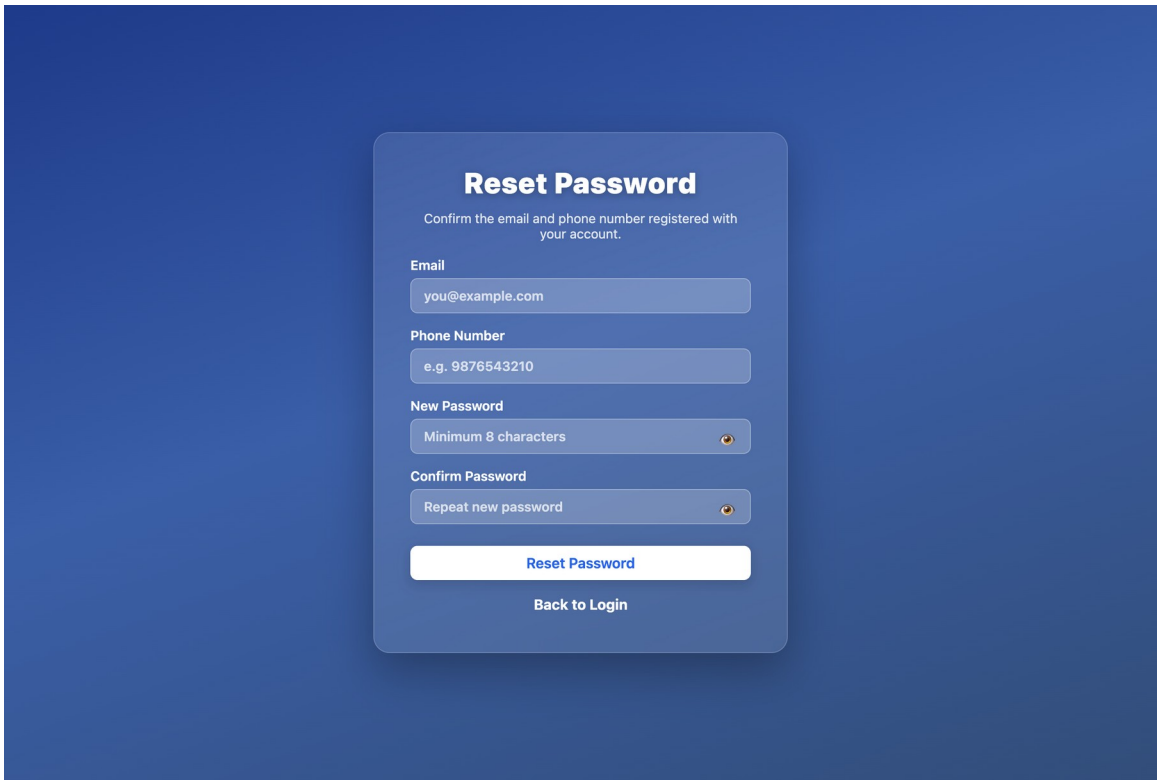


Figure 28: Reset Password Page

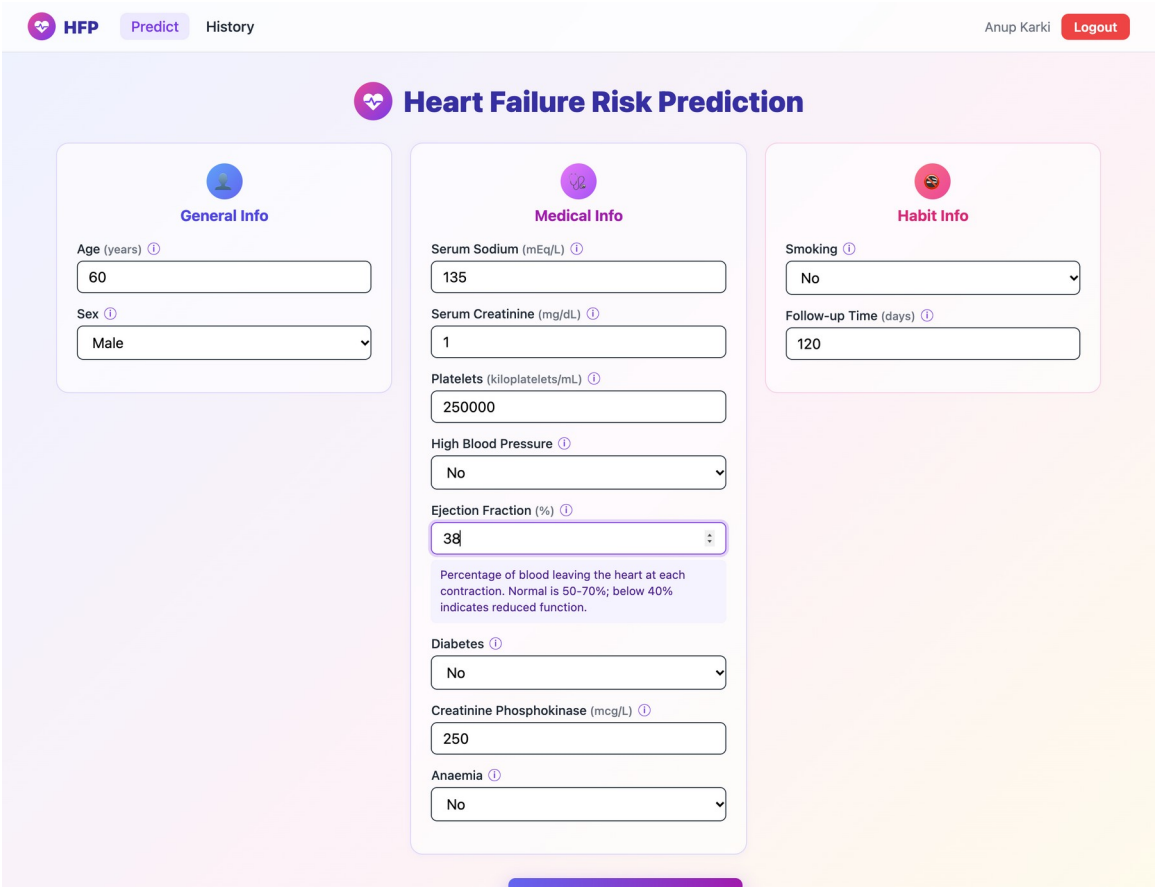


Figure 29: Heart Failure Risk Prediction Form with Field Description

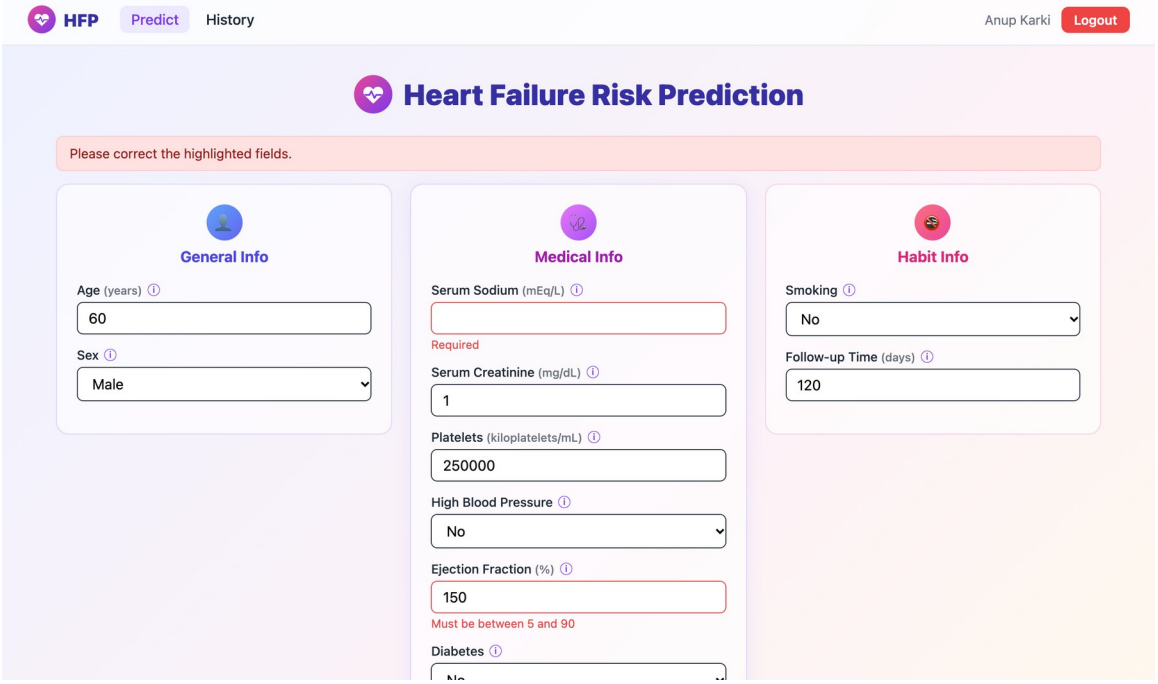


Figure 30: Validation of Invalid Input

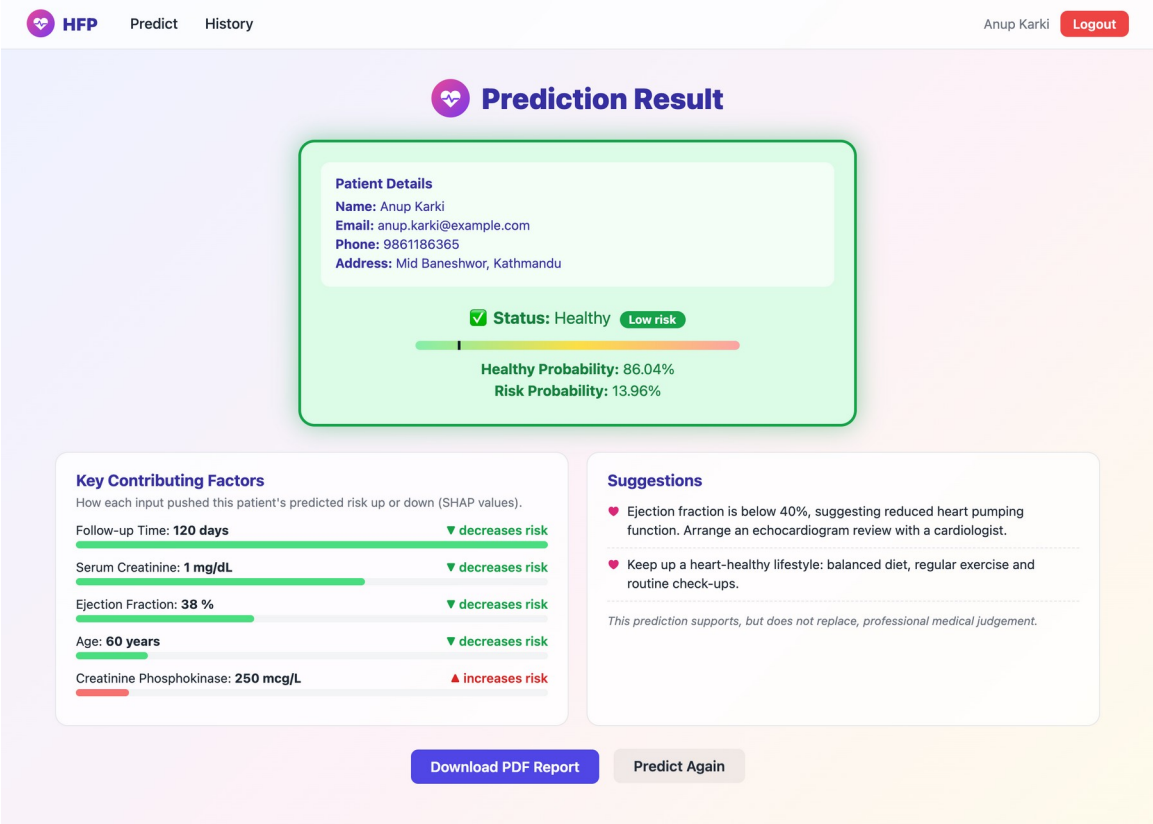


Figure 31: Prediction Result Page (Healthy Patient)

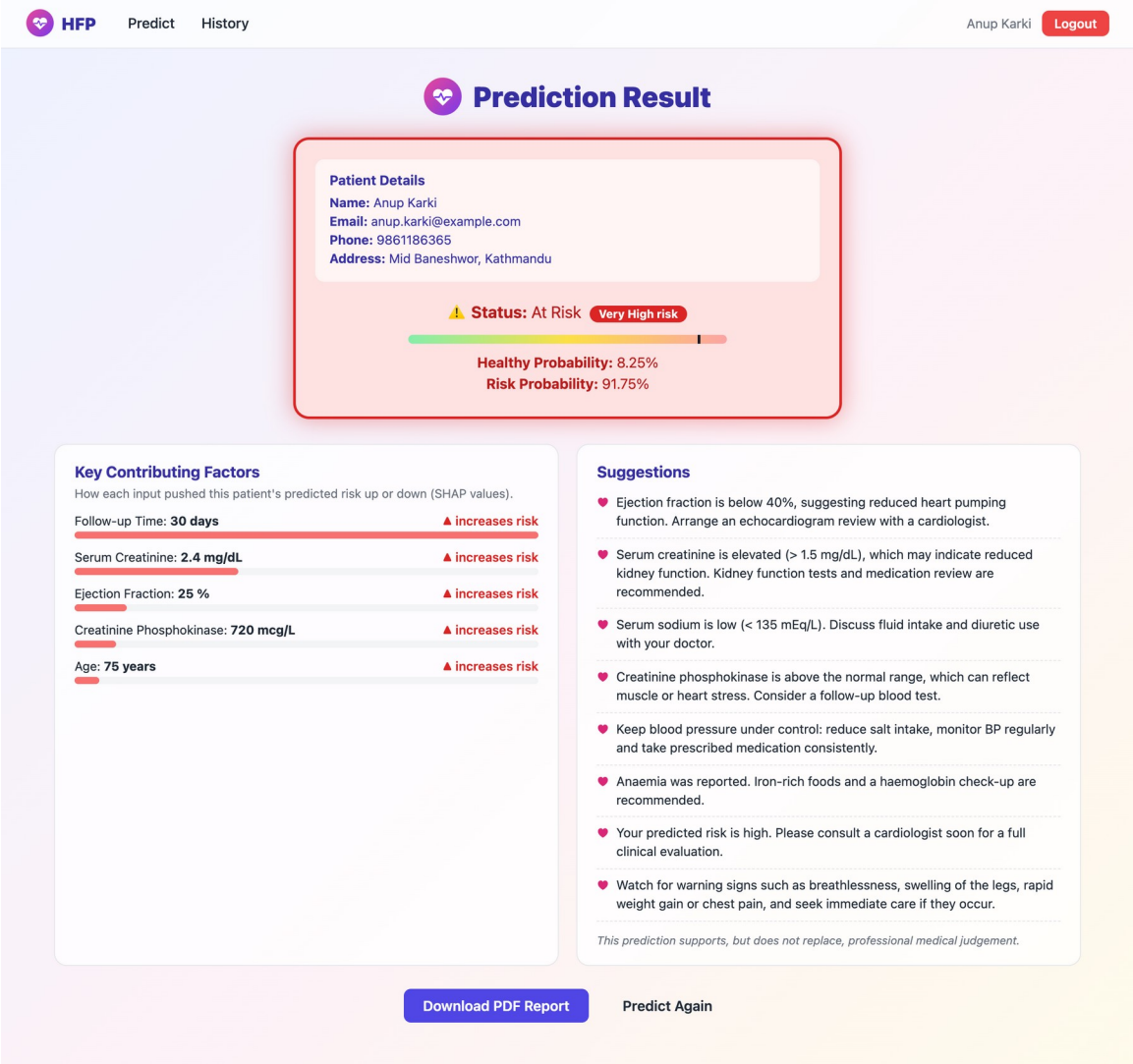


Figure 32: Prediction Result Page (At-Risk Patient) with Contributing Factors and Suggestions

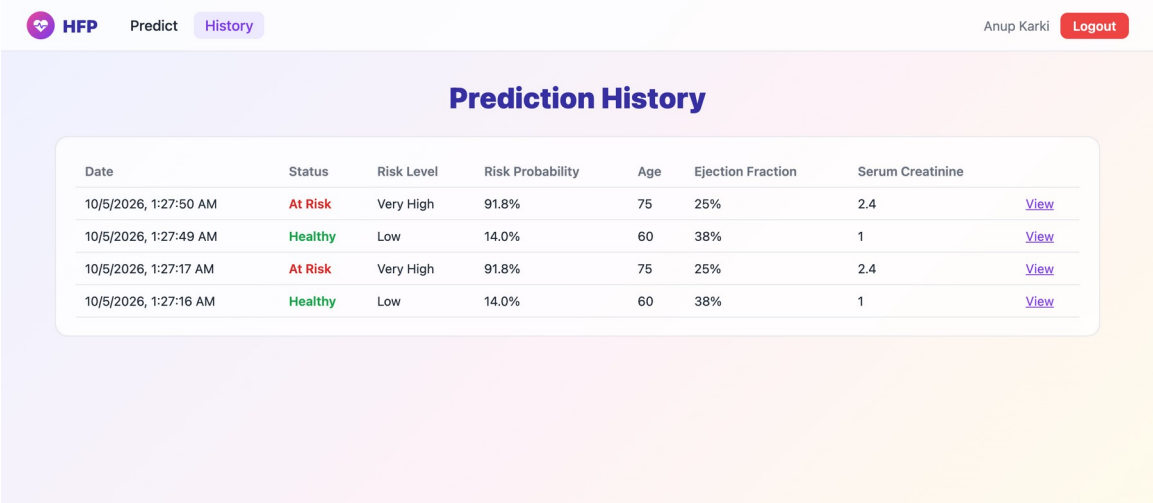
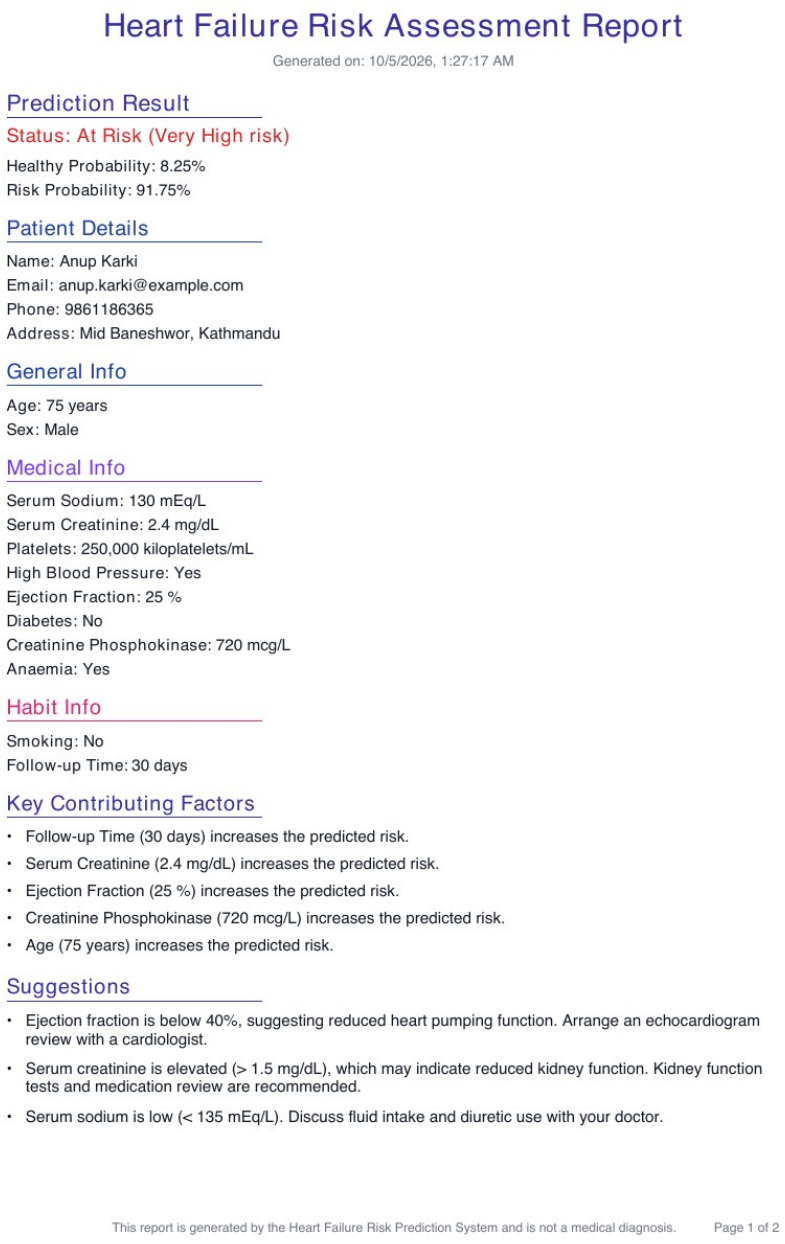


Figure 33: Prediction History Page



Heart Failure Risk Assessment Report

Generated on: 10/5/2026, 1:27:17 AM

Prediction Result

Status: At Risk (Very High risk)

Healthy Probability: 8.25%
Risk Probability: 91.75%

Patient Details

Name: Anup Karki
Email: anup.karki@example.com
Phone: 9861186365
Address: Mid Baneshwor, Kathmandu

General Info

Age: 75 years
Sex: Male

Medical Info

Serum Sodium: 130 mEq/L
Serum Creatinine: 2.4 mg/dL
Platelets: 250,000 kiloplatelets/mL
High Blood Pressure: Yes
Ejection Fraction: 25 %
Diabetes: No
Creatinine Phosphokinase: 720 mcg/L
Anaemia: Yes

Habit Info

Smoking: No
Follow-up Time: 30 days

Key Contributing Factors

- Follow-up Time (30 days) increases the predicted risk.
- Serum Creatinine (2.4 mg/dL) increases the predicted risk.
- Ejection Fraction (25 %) increases the predicted risk.
- Creatinine Phosphokinase (720 mcg/L) increases the predicted risk.
- Age (75 years) increases the predicted risk.

Suggestions

- Ejection fraction is below 40%, suggesting reduced heart pumping function. Arrange an echocardiogram review with a cardiologist.
- Serum creatinine is elevated (> 1.5 mg/dL), which may indicate reduced kidney function. Kidney function tests and medication review are recommended.
- Serum sodium is low (< 135 mEq/L). Discuss fluid intake and diuretic use with your doctor.

This report is generated by the Heart Failure Risk Prediction System and is not a medical diagnosis. Page 1 of 2

Figure 34: Downloaded PDF Report of a Patient

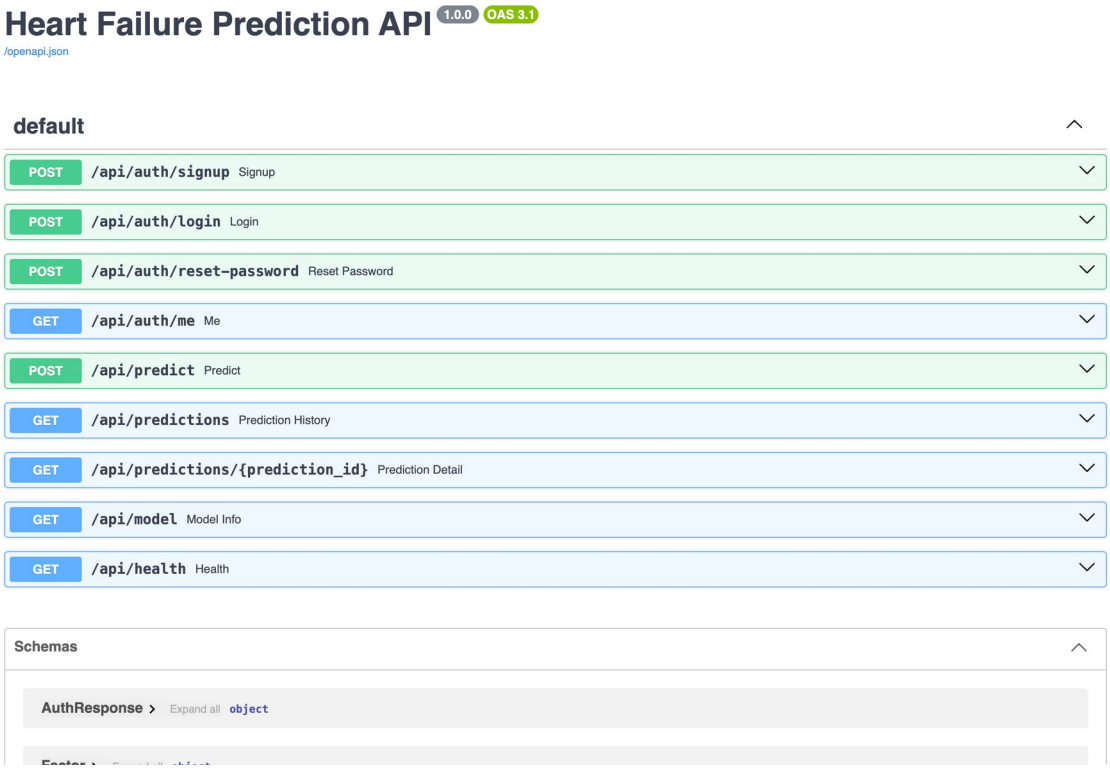


Figure 35: Interactive API Documentation of the Backend

Appendix B: Source Code

Selected source files of the system are listed below. The complete source code is available in the project repository.

B.1 Prediction Module (backend/app/predictor.py)

```
class Predictor:
    def __init__(self, model_path=None, meta_path=None):
        self.pipeline = joblib.load(model_path or config.ARTIFACTS_DIR /
"model.joblib")
        self.meta = json.loads((meta_path or config.ARTIFACTS_DIR /
"model_meta.json").read_text())
        self.features: list[str] = self.meta["features"]
        self.threshold: float = self.meta.get("threshold", 0.5)

    def _contributions(self, frame: pd.DataFrame) -> np.ndarray:
        """Per-feature contributions (log-odds) for one patient.

        XGBoost provides exact TreeSHAP values natively via ``pred_contribs``.
Linear models use
        coefficient x standardized value; other models fall back to global
importance signed by
        how far the input is from the training mean.
        """
        scaler = self.pipeline.named_steps["scaler"]
        model = self.pipeline.named_steps["model"]
        scaled = scaler.transform(frame)
        if hasattr(model, "get_booster"):
            import xgboost as xgb

            contribs = model.get_booster().predict(xgb.DMatrix(scaled,
feature_names=self.features), pred_contribs=True)
            return contribs[0, :-1]  # last column is the bias term
        if hasattr(model, "coef_"):
            return np.asarray(model.coef_).ravel() * scaled[0]
        importances = self.meta.get("feature_importances") or {f: 1 /
len(self.features) for f in self.features}
        return np.array([importances[f] for f in self.features]) * scaled[0]

    def predict(self, inputs: dict, top_k: int = 5) -> Prediction:
        frame = pd.DataFrame([[inputs[f] for f in self.features]],
columns=self.features)
        probability = float(self.pipeline.predict_proba(frame)[0, 1])

        contributions = self._contributions(frame)
        order = np.argsort(-np.abs(contributions))[:top_k]
        factors = [
            {
                "feature": self.features[i],
                "label": FEATURE_LABELS.get(self.features[i], self.features[i]),
                "value": float(inputs[self.features[i]]),
                "contribution": round(float(contributions[i]), 4),
                "direction": "increases" if contributions[i] > 0 else
"decreases",
            }
            for i in order
        ]
        return Prediction(
            probability_at_risk=probability,
            status="At Risk" if probability >= self.threshold else "Healthy",
            risk_level=risk_level(probability),
            factors=factors,
        )
```

B.2 Prediction Endpoint (backend/app/main.py)

```
@app.post("/api/predict", response_model=PredictionResult)
def predict(body: HealthData, user: CurrentUser):
    inputs = body.model_dump()
    result = get_predictor().predict(inputs)
    p_risk = round(result.probability_at_risk, 4)
    details = {"top_factors": result.factors, "suggestions":
build_suggestions(inputs, result.status)}

    with db.connect() as conn:
        health_id = conn.execute(
            "INSERT INTO health_data (user_id, inputs_json) VALUES (?, ?)",
(user.id, json.dumps(inputs))
        ).lastrowid
        pred_id = conn.execute(
            """INSERT INTO predictions
               (health_data_id, status, risk_level, probability_healthy,
probability_at_risk, details_json)
               VALUES (?, ?, ?, ?, ?, ?)""",
            (health_id, result.status, result.risk_level, round(1 - p_risk, 4),
p_risk, json.dumps(details)),
        ).lastrowid
        health = conn.execute("SELECT * FROM health_data WHERE id = ?",
(health_id,)).fetchone()
        pred = conn.execute("SELECT * FROM predictions WHERE id = ?",
(pred_id,)).fetchone()
    return _result_from_rows(health, pred)
```

B.3 Suggestion Module (backend/app/suggestions.py)

```
def build_suggestions(inputs: dict, status: str) -> list[str]:
    tips: list[str] = []

    if inputs["ejection_fraction"] < 40:
        tips.append(
            "Ejection fraction is below 40%, suggesting reduced heart pumping
function. "
            "Arrange an echocardiogram review with a cardiologist."
        )
    if inputs["serum_creatinine"] > 1.5:
        tips.append(
            "Serum creatinine is elevated (> 1.5 mg/dL), which may indicate
reduced kidney function. "
            "Kidney function tests and medication review are recommended."
        )
    if inputs["serum_sodium"] < 135:
        tips.append(
            "Serum sodium is low (< 135 mEq/L). Discuss fluid intake and
diuretic use with your doctor."
        )
    if inputs["creatinine_phosphokinase"] > 582:
        tips.append(
            "Creatinine phosphokinase is above the normal range, which can
reflect muscle or heart stress. "
            "Consider a follow-up blood test."
        )
    if not 150_000 <= inputs["platelets"] <= 450_000:
        tips.append("Platelet count is outside the normal range (150,000-
450,000). A blood work-up is advised.")
    if inputs["high_blood_pressure"]:
        tips.append(
            "Keep blood pressure under control: reduce salt intake, monitor BP
regularly and take "
            "prescribed medication consistently."
        )
    if inputs["diabetes"]:
```

```
        tips.append("Maintain good blood sugar control through diet, activity
and regular HbA1c monitoring.")
    if inputs["anaemia"]:
        tips.append("Anaemia was reported. Iron-rich foods and a haemoglobin
check-up are recommended.")
    if inputs["smoking"]:
        tips.append("Quitting smoking is one of the most effective ways to
reduce heart failure risk.")

    if status == "At Risk":
        tips.append(
            "Your predicted risk is high. Please consult a cardiologist soon for
a full clinical evaluation."
        )
        tips.append(
            "Watch for warning signs such as breathlessness, swelling of the
legs, rapid weight gain or chest "
            "pain, and seek immediate care if they occur."
        )
    else:
        tips.append("Keep up a heart-healthy lifestyle: balanced diet, regular
exercise and routine check-ups.")

    return tips
```

B.4 Model Definitions (ml/train.py)

```
def build_models(scale_pos_weight: float) -> dict[str, object]:
    return {
        "logistic_regression": LogisticRegression(max_iter=1000,
random_state=SEED),
        "svm": CalibratedClassifierCV(SVC(kernel="rbf", random_state=SEED),
ensemble=False),
        "decision_tree": DecisionTreeClassifier(max_depth=5,
min_samples_leaf=10, random_state=SEED),
        "random_forest": RandomForestClassifier(
            n_estimators=300, max_depth=8, min_samples_leaf=5,
class_weight="balanced", random_state=SEED, n_jobs=-1
        ),
        "xgboost": XGBClassifier(
            n_estimators=300,
            max_depth=4,
            learning_rate=0.05,
            subsample=0.8,
            colsample_bytree=0.8,
            reg_lambda=1.0,
            scale_pos_weight=scale_pos_weight,
            eval_metric="logloss",
            random_state=SEED,
            n_jobs=-1,
        ),
        "gradient_boosting": GradientBoostingClassifier(n_estimators=200,
max_depth=3, random_state=SEED),
    }


# Small grid tuned with group-aware CV on the training split only (scored on F1,
since
# balanced precision/recall matters more than raw accuracy for this medical
task).
XGB_GRID = {
    "model__max_depth": [2, 3, 4],
    "model__n_estimators": [150, 300],
    "model__learning_rate": [0.03, 0.1],
    "model__min_child_weight": [1, 5],
}
```

B.5 Prediction Form Submission (frontend/src/pages/PredictionForm.jsx)

```
  const handleSubmit = async (e) => {
    e.preventDefault()
    setError('')
    const found = validate(formData)
    setErrors(found)
    if (Object.keys(found).length) {
      setError('Please correct the highlighted fields.')
      return
    }
    setLoading(true)
    try {
      const body = Object.fromEntries(Object.entries(formData).map(([k, v]) =>
[k, Number(v)]))
      const result = await api('/predict', { method: 'POST', body })
      navigate(`/result/${result.id}`, { state: { result } })
    } catch (err) {
      setError(err.message)
      setErrors(err.fieldErrors || {})
    } finally {
      setLoading(false)
    }
  }
```